

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK

BUSINESS INTELLIGENCE

Chapter 11

AI Programming Languages

Asst.Prof. Sakchai Tangprasert (Ph.D. in IT)

Sakchai.t@sci.kmutnb.ac.th

Analytics and Big Data for Accountants

By Jim Lindell

© 2017 Association of International Certified Professional Accountants, Inc.

1. Top 10 AI Programming Languages

1. Python
2. R
3. Julia
4. Scala
5. Java
6. C++
7. Javascript
8. Lisp
9. Haskell
10. Mojo

- อ้างอิง: www.datacamp.com, Jun 26, 2024

2. Python

- Python เป็นภาษาการเรียนรู้ของเครื่อง (Machine Learning) ที่ได้รับความนิยมและดีที่สุด โดยกว่า 60% ของนักพัฒนาการเรียนรู้ของเครื่อง (Machine Learning) ใช้และจัดลำดับความสำคัญในการพัฒนา มีประเด็นสำคัญบางประการที่ทำให้ Python น่าสนใจมาก ประการหนึ่ง การเรียนรู้เป็นเรื่องง่าย ซึ่งเป็นสิ่งสำคัญสำหรับผู้ที่ต้องการเริ่มต้นใช้งานการเรียนรู้ของเครื่อง (Machine Learning) นอกจากนี้ยังปรับขนาดได้และเป็น Open Source
- Python เป็นตัวเลือกอันดับต้น ๆ สำหรับผู้ที่ต้องการทำงานที่เกี่ยวข้องกับการวิเคราะห์ข้อมูล วิทยาการข้อมูล การเรียนรู้ของเครื่อง และ AI เนื่องจากมีระบบนิเวศของห้องสมุดขนาดใหญ่ที่มี Library ต่าง ๆ มากมายสำหรับจัดการ แปลง และประมวลผลข้อมูล Library
- Library สำหรับ Python ได้แก่ Numpy, Pandas, Matplotlib, Seaborn และ sci-kit Learn เป็นต้น
- ประโยชน์หลักของ Python เช่น
  - ใช้งานง่าย
  - เป็น Open Source
  - มีความยืดหยุ่น

3. R

- ภาษา R ซึ่งผู้ที่ไม่ใช่โปรแกรมเมอร์และโปรแกรมเมอร์สามารถใช้ได้ ผู้ที่ไม่ใช่โปรแกรมเมอร์ เช่น นักขุดข้อมูล นักวิเคราะห์ข้อมูล และนักสถิติพบว่า R มีประโยชน์มากมาย
- ภาษา R เป็นตัวเลือกอันดับต้นสำหรับการประมวลผลจำนวนมาก และเป็นภาษาที่ใช้สำหรับแอปพลิเคชันการเรียนรู้ของเครื่อง (Machine Learning) ที่ใช้ข้อมูลทางสถิติจำนวนมาก IDEs และเครื่องมือที่เป็นมิตรกับผู้ใช้ช่วยให้สามารถวาดกราฟและจัดการ Library ได้ นอกจากนี้ยังมีเครื่องมือที่หลากหลายในการฝึกอบรมและประเมินอัลกอริทึมการเรียนรู้ของเครื่องสำหรับการทำนายเหตุการณ์ในอนาคต
- ภาษา R ถูกนำมาใช้อย่างสูงในด้านสถิติทางวิศวกรรมชีวภาพและชีวการแพทย์ แต่ก็เป็นที่นิยมสำหรับการนำการเรียนรู้ของเครื่องไปใช้ เช่น การจำแนกประเภท การถดถอย และการสร้างแผนผังการตัดสินใจ
- ประโยชน์หลักของ R เช่น
  - ภาษาโปรแกรม Open Source
  - ให้เทคนิคการเรียนรู้ของเครื่องที่หลากหลาย
  - รูปแบบการเขียนโปรแกรมอย่างง่าย
  - มีความยืดหยุ่นสูงและใช้งานข้าม Platform ได้

4. Java

- ภาษา Java เป็นหนึ่งในภาษาโปรแกรมที่เป็นที่รู้จักมากที่สุดสำหรับการเรียนรู้ของเครื่อง Java มีความแข็งแกร่งในอุตสาหกรรม เป็นที่นิยมโดยเฉพาะในหมู่วิศวกรที่มีพื้นฐานการพัฒนา Java เนื่องจากพวกเขาไม่จำเป็นต้องเรียนรู้ภาษาโปรแกรมใหม่
- ภาษา Java มีอาร์เรย์ของคุณสมบัติที่ทำให้เป็นตัวเลือกที่ยอดเยี่ยม เช่น การใช้งานง่าย การโต้ตอบกับผู้ใช้ที่ดีขึ้น บริการแพ็คเกจ การแก้ Bug ที่ง่าย และการแสดงข้อมูลแบบ Graphic มี Library มากมายสำหรับการเรียนรู้ของเครื่อง เช่น Weka, JavaML, Deeplearning4j และ ELKI นอกจากนี้ยังมี Java Virtual Machine ซึ่งเป็นหนึ่งใน Platform ที่ดีที่สุดสำหรับการเรียนรู้ของเครื่อง เนื่องจากช่วยให้วิศวกรสามารถเขียน Code เดียวกันบนหลาย Platform ได้
- ประโยชน์หลักของ Java เช่น
  - Library มากมายสำหรับการเรียนรู้ของเครื่อง
  - Scalability
  - Java Virtual Machine
  - ง่ายสำหรับวิศวกรที่มีพื้นฐานการพัฒนา Java

5. Julia

- Julia เป็นภาษาการเขียนโปรแกรม Dynamic ที่มีประสิทธิภาพสูงและมีวัตถุประสงค์ทั่วไป กลายเป็นคู่แข่งที่มีศักยภาพสำหรับ Python และ R ภาษานี้มีคุณลักษณะที่ยอดเยี่ยมมากมายสำหรับการเรียนรู้ของเครื่อง (Machine Learning) โดยเฉพาะ และเป็นภาษาที่ใช้งานทั่วไปสำหรับการพัฒนาในวงกว้าง การใช้งานที่หลากหลาย เหมาะที่สุดสำหรับการวิเคราะห์เชิงตัวเลขประสิทธิภาพสูงและวิทยาการคำนวณ
- Julia รองรับฮาร์ดแวร์ทุกประเภท เช่น TPU และ GPU และใช้งานโดยบริษัทขนาดใหญ่อย่าง Apple, Oracle, Disney และ NASA
- ประโยชน์หลักของ Julia เช่น
  - เรียกใช้งานได้ในระดับสากล
  - สามารถปรับขนาดได้
  - คุณสมบัติมากมายสำหรับการเรียนรู้ของเครื่อง (Machine Learning) โดยเฉพาะ

6. Lisp

- Lisp Programming Language เป็นภาษาคอมพิวเตอร์ขั้นสูง นอกจากนำไปใช้ในการเขียนโปรแกรมทั่วไปแล้วยังสามารถใช้ได้ดีในการประมวลผลสัญลักษณ์ ดังนั้นจึงถูกใช้อย่างแพร่หลายทางด้านปัญญาประดิษฐ์ นอกจากนี้ ในภาษา Lisp ไม่จำเป็นต้องประกาศชนิดตัวแปรที่ใช้ในโปรแกรม ดังนั้นจึงง่ายในการเขียนและเรียนรู้
- ในปัจจุบัน LISP มักใช้สำหรับปัญหา Logic เชิงอุปนัยและการเรียนรู้ของเครื่อง เป็นที่รู้จักอย่างกว้างขวางในด้านการสร้าง Chatbot AI ตัวแรก และยังคงสามารถใช้เพื่อพัฒนา Chatbot สำหรับอุตสาหกรรมต่าง ๆ เช่น E-commerce ได้จนถึงทุกวันนี้
- ประโยชน์หลักของ Lisp เช่น
  - ความสามารถในการสร้างต้นแบบอย่างรวดเร็ว
  - ยืดหยุ่นและมีประสิทธิภาพสูง
  - ปรับให้เข้ากับ Solution ที่ Programmer กำลังเขียน Code
  - รองรับการแสดงออกเชิงสัญลักษณ์

7. C++

- C++ ภาษาโปรแกรมที่มีประสิทธิภาพสูงและมีความยืดหยุ่นสำหรับการพัฒนาโปรแกรมที่ใช้ปัญญาประดิษฐ์ (Artificial Intelligence: AI) อย่างมาก ซึ่งทำให้เป็นทางเลือกที่ดีสำหรับการพัฒนาโปรแกรมปัญญาประดิษฐ์ (Artificial Intelligence: AI) ที่ ต้องการประสิทธิภาพของระบบ (System Performance) และการจัดการทรัพยากร (Resource Management) อย่างมีประสิทธิภาพสูงสุด
- ประโยชน์หลักของ C++ เช่น
  - ประสิทธิภาพสูง: C++ เป็นภาษาที่มีประสิทธิภาพสูงที่สุดในการประมวลผลข้อมูล ซึ่งเหมาะสำหรับการพัฒนาโปรแกรม AI ที่ต้องการประมวลผลข้อมูลมากมายอย่างรวดเร็ว
  - ความยืดหยุ่น: C++ มีความยืดหยุ่นสูงที่สุดในการจัดการหน่วยความจำและการควบคุมการทำงานของระบบ ทำให้เป็นทางเลือกที่เหมาะสำหรับการพัฒนาปัญญาประดิษฐ์ (Artificial Intelligence: AI) ที่ต้องการการควบคุมทรัพยากรอย่างมาก

8. โครงสร้างข้อมูลแบบลำดับรายการของ Python

- โครงสร้างข้อมูลแบบลำดับรายการของ Python ถึงแม้จะมีไลบรารีเกี่ยวกับ Data Science โดยตรง แต่ในบางกรณีต้องนำโครงสร้างข้อมูลในแบบของ Python มาใช้งานร่วมด้วย เพื่อเป็นพื้นฐานสำหรับการใช้งานในลำดับต่อๆ ไป ซึ่งมีรายละเอียดดังนี้

9. การกำหนดช่วงข้อมูลด้วย range()

- ฟังก์ชัน range() ใช้ในการกำหนดช่วงหรือขอบเขตของข้อมูลชนิดเลขจำนวนเต็มที่มีค่าต่อเนื่อง เช่นระหว่าง 1-10 หรือจาก 100 -999 เป็นต้น ซึ่งมีรูปแบบดังนี้

range(stop)

range(start, stop)

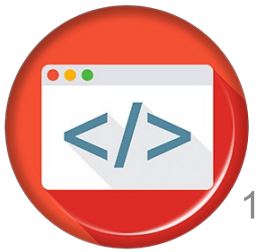
range(start, stop, step)

- start คือค่าเริ่มต้น ซึ่งต้องเป็นจำนวนเต็ม (ติดลบได้)
- stop คือค่าขอบเขตบนของช่วงข้อมูล แต่ไม่รวมถึงตัวเองมัน เช่น หากกำหนด stop เป็น 10 แสดงว่าช่วงของข้อมูลจะสิ้นสุดแค่ 9 หรือเท่ากับ stop 1 นั่นเอง
- step คือระยะห่างของการเพิ่มหรือลดข้อมูล ทั้งหากใช้ฟังก์ชัน range() แบบไม่กำหนดค่า step ช่วงข้อมูลจะเพิ่มครั้งละ 1 หรือค่า step = 1 โดยอัตโนมัตินั่นเอง
- ถ้าระบุเฉพาะ stop เพียงค่าเดียว จะหมายถึงค่า start = 0 เช่น หากกำหนดเป็น range(10) จะเทียบเท่ากับ range(0, 10) เป็นต้น

9. การกำหนดช่วงข้อมูลด้วย range() ต่อ

- ถ้าระบุเฉพาะ stop เพียงค่าเดียว จะหมายถึงค่า start = 0 เช่น หากกำหนดเป็น range(10) จะเทียบเท่ากับ range(0, 10) เป็นต้น

```
one_to_ten = range(1, 11)
print(*one_to_ten)                    #1 2 3 4 5 6 7 8 9 10
even_numbers = range(0, 20, 2)
print(*even_numbers)                  # 0 2 4 6 8 10 12 14 16 18
count_down = range(10, 0, -1)
print(*count_down)                    #10 9 8 7 6 5 4 3 2 1
```



9. การกำหนดช่วงข้อมูลด้วย range() ต่อ

- print ("variable) ใช้สำหรับการพิมพ์หรือแสดงข้อมูลแบบลำดับรายการ ซึ่งเทียบเท่ากับการใช้ลูป for เช่น

```
print(*one_to_ten)
#เทียบเท่ากับ
for x in one_to_ten: print(x, sep=' ', end=' ')
```

10. รายการข้อมูลแบบ List

- List เป็นการจัดเก็บข้อมูลด้วยตัวแปรเดียวกัน เพื่อเข้าถึงข้อมูลเหล่านั้นด้วยเลขลำดับ หรือเทียบ เท่ากับอาร์เรย์ในภาษาอื่นๆ นั่นเอง ซึ่งสมาชิกของลิสต์อาจเป็นข้อมูลชนิดเดียวกันหรือต่างชนิดกันได้ โดยลักษณะที่สำคัญเกี่ยวกับลิสต์มีดังนี้
- การสร้างลิสต์ อาจใช้ฟังก์ชัน list() หรือวงเล็บ [ ] เช่น

```
a = list([1, 2, 3, 4, 5])          #ลิสต์ที่มีสมาชิก 5 ตัวคือ 1, 2, 3, 4, 5
b = list(range(3, 6))              #สร้างลิสต์โดยใช้ range() ซึ่งมีสมาชิกเป็น 3, 4, 5
c = list(1, True, 'Three', 4.5)    #สมาชิกของลิสต์อาจเป็นข้อมูลต่างชนิดกันก็ได้
d = list('abcd')                   #ลิสต์ที่มีสมาชิก 4 ตัวคือ 'a', 'b', 'c', 'd'
e = [1, 2, 3, 4, 5]
f = [range(3, 6)]
g = [1, True, 'Three', 4.5]
```

10. รายการข้อมูลแบบ List (ต่อ)

- การเข้าถึงสมาชิกแต่ละตัวของลิสต์จะกำหนดเลขลำดับไว้ในวงเล็บ [ ] หรือใช้ลูป for เช่นเดียวกับ range() ดังนี้

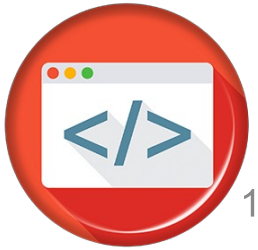
```
data = [1, 2, 3, 4, 5]
print(data[0])                          #1
print(data[3])                          #4
print(*data)                            #1 2 3 4 5
for x in data: print(x, sep= ‘)         #1 2 3 4 5
```

10. รายการข้อมูลแบบ List (ต่อ)

```
data = [1, 2, 3, 4, 5]
print(data[0])
print(data[3])

print(*data)

for x in data: print(x, sep=' ', end=' ')
```



10. รายการข้อมูลแบบ List (ต่อ)

- สามารถนำโอเปอเรเตอร์บางชนิดมาใช้ร่วมกับลิสต์ได้ คือ

list1 + list2 และ list1 += list2	เชื่อมต่อสมาชิกจากทั้ง 2 ลิสต์เข้าด้วยกัน แต่ข้อควรระวังคือ ลำดับของสมาชิกที่ได้จาก list1 + list2 อาจแตกต่างไปจาก list2 + list1
list * n หรือ n * list	คัดลอกสมาชิกทั้งหมดเป็นจำนวน n ครั้ง แล้วนำไปเชื่อมต่อเข้ากับลิสต์เดิม
x **in** list	ตรวจสอบว่ามีสมาชิกค่า x อยู่ในลิสต์หรือไม่ ถ้ามีจะคืนค่า True
x **not in** list	ตรวจสอบว่า **ไม่** มีสมาชิกค่า x อยู่ในลิสต์ใช่หรือไม่ ถ้าใช่จะคืนค่า True
==, !=	เปรียบเทียบลิสต์ว่าเท่ากันหรือไม่
list[start : stop]	คัดลอกสมาชิกของลิสต์เริ่มจากลำดับ start ไปจนถึงก่อนลำดับ stop
list[start :]	คัดลอกสมาชิกของลิสต์เริ่มจากลำดับ start ไปจนถึงตัวสุดท้าย
list[: stop]	คัดลอกสมาชิกของลิสต์เริ่มตั้งแต่ตัวแรกไปจนถึงก่อนลำดับ stop
list[start : stop : step]	คัดลอกสมาชิกของลิสต์เริ่มตั้งแต่ตัวแรกไปจนถึงก่อนลำดับ stop โดยมีช่วงห่างเท่ากับค่า step

10. รายการข้อมูลแบบ List (ต่อ)

```
a = [1, 3, 5]
b = [2, 4, 6, 8]
c = a + b                    #c = [1, 3, 5, 2, 4, 6, 8]
d = b + a                    #d = [2,4,8,1,3, 1,5]
e = [1, 3, 5, 0]
print(e * 2)                 #[1,3,5, 5, 0, 1, 3, 5, 0]
f = [0, 1, 2, 3, 4, 5, 6, 7]
print(f[1:3])                #[1, 2]
print(f[4:10])               #[4, 5, 6, 7]  ถ้าไม่ครบ จะคัดลอกเท่าที่มี
print(f[3:])                 #[3, 4, 5, 6, 7]
print(f[:5])                 #[0, 1, 2, 3, 4]
print(f[1:6:2])              #[1,3,5]
```

10. รายการข้อมูลแบบ List (ต่อ)

- ในภาษา Python มีฟังก์ชันแบบ Built-in ที่สามารถนำมาใช้ร่วมกับลิสต์ได้คือ

```
a = [1, 3, 5, 7, 9]
print(len(a))                    #5
print(sum(a))                    #25
b = [108, 1009, 101, 7, 11]
print(max(b))                    #1009
print(min(b))                    #7
c = [108, 1009, 7, 11]
for (i, v) in enumerate(c):
        print(f'(i)-(v)')       #0-108,11009,2 7,3-11
nums = [1, 2, 4, 5]
del nums[2]                      #nums = [1, 2, 5]
```

11. List แบบ 2 มิติ

- กรณีที่สมาชิกแต่ละตัวของลิสต์ก็เป็นลิสต์ด้วย จะเรียกว่า ลิสต์แบบ 2 มิติ (Two-Dimensional List) เช่นกรณีต่อไปนี้

```
nums = [[0,1,2,3], [4,5,6,7], [8,9]]
print(nums[0])                  #[0,1,2,3]
print(nums[1])                  #[4,5,6,7]
print(len(nums))                #3
print(len(nums[0]))             #4
print(len(nums[2]))             #2
colors = [['red', green',,'blue'], ['black',,'white']]
print(colors[2])                # [ ' yellow' ]
print(len(colors))              #2
print(len(colors[1]))           #2
```

11. List แบบ 2 มิติ (ต่อ)

- เนื่องจากมีลิสต์ซ้อนกัน 2 ชั้น ดังนั้นการอ้างถึงสมาชิกย่อย ๆ ต้องใช้วงเล็บ [] จำนวน 2 อัน ในรูปแบบ

[ลำดับของลิสต์ย่อย] [ลำดับของสมาชิกภายในลิสต์ย่อย)

- สำหรับแนวทางการใช้งานมีดังนี้

```
colors = [['red','green', 'blue'], ['black', 'white']]
print(colors[0][0])          #red
print(colors[0][1])          #green
print(colors[1][0])          #black
print(colors[1][1])          #white
```

11. List แบบ 2 มิติ (ต่อ)

- หากจะเข้าถึงสมาชิกย่อย ๆ ของลิสต์ 2 มิติด้วยลูปแบบ for-in ก็ต้องใช้ลูปซ้อนกัน โดยที่
- ลูปชั้นนอก ใช้อ้างถึงลำดับของลิสต์ย่อย ๆ
- ลูปชั้นใน ใช้อ้างถึงลำดับของสมาชิกภายในลิสต์ย่อย ๆ

```
colors = [['red','green', 'blue'], ['black','white']]
for cols in colors:
        for c in cols:
                print(c, end=' ') #red green blue black white
```

11. List แบบ 2 มิติ (ต่อ)

- นอกจากนี้ การใช้ฟังก์ชันต่างๆ ของลิสต์ที่เกี่ยวกับการเข้าถึงสมาชิกแต่ละตัว จะต้องอ้างถึงลำดับของลิสต์ย่อยด้วย หรือถ้าจะเข้าถึงทั้งหมดก็อาจใช้ร่วมกับลูป เช่น

```
colors = [['red', 'green', 'blue'], ['black', 'white']]
colors[1].append ( 'yellow')
colors[1] += ['pink', 'purple']
print(colors[1])                    #['black', 'white','yellow', 'pink', 'purple']
if 'green' in colors[0]: print('yes') #ตรวจสอบเฉพาะสมาชิกในกลุ่มแรก
for cols in colors:
        if 'blue' in cols: print('yes')
```

12. รายการข้อมูลแบบ Tuple

- Tuple คือการจัดเก็บกลุ่มข้อมูลที่มีหลายๆ ค่า ด้วยตัวแปรเดียวกันในแบบลำดับรายการ ดังนั้น ทูเพิลจึงอาจดูคล้ายกับลิสต์ แต่ความจริงแล้วข้อกำหนดหลักๆ ของทูเพิลจะต่างจากลิสต์ กล่าวคือ ทูเพิลจะกำหนดสมาชิกตัวยวงเล็บ () นอกจากนี้ ค่าของสมาชิกจะคงที่ ซึ่งไม่สามารถแก้ไข ลบ หรือเปลี่ยนแปลงลำดับได้ สำหรับแนวทางการใช้ทูเพิลมีดังนี้
- การสร้าง Tuple อาจใช้วงเล็บ () หรือฟังก์ชัน tuple() ซึ่งสมาชิกของพิลอาจเป็นข้อมูลต่างชนิดกันก็ได้เช่นเดียวกับลิสต์

```
a = (1 , 2, 3)
list = [4, 5, 6]
b = tuple (list)          #สร้างทูเพิลจากลิสต์
c = tuple([7, 8, 9])      #สร้างทูเพิลจากลิสต์
d = (1,)                   # ถ้ามีสมาชิกตัวเดียว ให้วาง , ต่อท้าย
```

12. รายการข้อมูลแบบ Tuple (ต่อ)

- การอ้างถึงสมาชิก ต้องใช้วงเล็บ [ ] พร้อมระบุเลขลำดับเช่นเดียวกับลิสต์ ดังนี้

```
nums = (2, 4, 6, 8, 10)
print(nums[0])     #2
print(nums[1])     #4
print(*nums)       #2 4 6 8 10
nums[1] = 0        #Error เพราะสมาชิกของทูเพิลจะเปลี่ยนแปลงไม่ได้
```

13. รายการข้อมูลแบบ Dictionary

- Dictionary คือการจัดเก็บข้อมูลแบบรายการที่สมาชิกแต่ละตัวจะมีองค์ประกอบ 2 อย่างคือ key และ value หากจะเข้าถึงสมาชิกต้องใช้ key เป็นตัวกำหนด โดยรายละเอียดของดิกชันนารี มีดังต่อไปนี้
- การกำหนดสมาชิกของดิกชันนารี จะใช้วงเล็บ { } เพื่อกำหนด key และ value ดังแนวทางต่อไปนี้

```
countries = {'th':'Thailand', 'jp':'Jan', 'kr':'Korea’}
info = {'name':'John', 'age':30, 'height':70.5, 'single’:True}
ranges = {'a':range(80,100), 'b':range(70 9), 'c : range(60,9)}
odd_evens = {'odd :[1,3,5,5,7,9], 'even':[2,4,6,4,8]}
```

13. รายการข้อมูลแบบ Dictionary (ต่อ)

- หากต้องการอ้างถึงสมาชิกตัวใดของดิกชันนารี ต้องระบุผ่านคีย์ของสมาชิกตัวนั้นด้วยวงเล็บ [ ]แต่หากค่าของสมาชิกเป็นแบบลำดับรายการ เช่น ลิสต์ ทูเพิล หรือ range ต้องเพิ่มวงเล็บ []อีกอันสำหรับระบุลำดับในรายการ เช่น

```
countries = ('th':'Thailand', 'jp': Japan', kr':'Korea')
print(countries['th’])                    #Thailand
print(countries['jp’])                    #Japan
odd_evens = ('odd':[1,3,5,5,7,9], 'even':[2,4,6,8])
a = odd_evens['ood'][0]                   #a = 1
b = odd_evens['even'][3]                  #d = 8
```

13. รายการข้อมูลแบบ Dictionary (ต่อ)

- ในกรณีที่ต้องการเข้าถึงสมาชิกด้วยลูป for-in ก็อาจใช้แนวทางดังนี้

```
countries = {'th': Thailand', 'jp': Japan', 'kr':'Korea’}
for key in countries:          หรือใช้ countries.keys()
print(countries[key])
```

13. รายการข้อมูลแบบ Dictionary (ต่อ)

- ดิกชันนารีมีฟังก์ชันเฉพาะตัวสำหรับเข้าถึงและจัดการกับสมาชิก คือ
  - keys( ) อ่านคีย์ทั้งหมด โดยคืนค่ากลับมาเป็นรายการแบบทูเพิล
  - values() อ่านค่าทั้งหมด โดยคืนค่ากลับมาเป็นรายการแบบพิล
  - items() อ่านรายการทั้งหมด โดยคืนค่ากลับมาเป็นรายการแบบ tuple (key, value)

```
countries = ('th':'Thailand', 'jp':'Japan', 'kr':'Korea')
for key in countries.keys(): #หรือใช้ for key in countries:
print(countries[key])
values = countries.values()
for v in values: print(v, ' ', end='')
for (k, v) in countries.items(): print(f'(k): (v)')
```

14. การใช้ไลบรารี NumPy

- NumPy มาจากคำว่า Numerical Python เป็นไลบรารีสำหรับจัดการข้อมูลประเภทตัวเลข โดย NumPy จะจัดการข้อมูลในแบบอาร์เรย์ (Array) หรือเรียกว่า Arrav-Orented เป็นหลัก ถึงแม้จะมีลักษณะคล้ายกับลิสต์ (List) ในภาษา Python แต่อาร์เรย์ของ NumPy จะทำงานได้เร็วกว่าลิสต์ นอกจากนี้ NumPy ยังประกอบด้วย ฟังก์ชันต่างๆ สำหรับการประมวลผลทางคณิตศาสตร์ให้เลือกใช้มากมาย ซึ่งช่วยลดความจำเป็นการเขียน Code ของ Python ลงไปได้
- อย่างไรก็ตาม NumPy ไม่ใช่องค์ประกอบที่เป็นส่วนหนึ่งของภาษา Python แต่เป็นไลบรารีเสริมจากภายนอก ซึ่งหากติดตั้งชุดเครื่องมือด้วย Anaconda ก็จะมีไลบรารี NumPy ถูกติดตั้งเอาไว้ด้วย ดังนั้น จึงสามารถใช้งานผ่าน Jupyter ได้ทันที
- หากต้องการใช้งาน NumPy สิ่งที่ต้องทำเป็นลำดับแรกคือ การระบุคำสั่ง import เพื่ออ้างอิงไลบรารี NumPy ดังภาพ สำหรับ np คือชื่อ (alias) ที่ใช้อ้างอิงแทน NumPy ซึ่งส่วนใหญ่นิยมใช้ชื่อแบบนี้ หรือจะตั้งชื่อเป็นอย่างอื่นก็ได้ ส่วนแนวทางการใช้โลบรารี NumPy มีลักษณะที่น่าสนใจดังนี้

15. การสร้างอาร์เรย์

- การสร้างอาร์เรย์ ก็คือการกำหนดข้อมูลในแบบแบบของ NumPy นั่นเอง ซึ่งมีหลายวิธีให้เลือกใช้งาน คือ
- สร้างจากข้อมูลประเภทรายการ (Sequence) ของ Python เช่น ลิสต์ ทูเพิล หรือ Range ด้วย
- เมธอด np.array () เช่น

```
a = np.array([2, 4, 6])                    #สร้างอาร์เรย์จากลิสต์
b = np.array(range(1, 11))                 #สร้างอาร์เรย์จากฟังก์ชัน range ( )
c=np.array((7,11,101,555))                 #สร้างอาร์เรย์จากทูเพิล
d = np.array([3 * x for x in range(1, 6)])
```

15. การสร้างอาร์เรย์ (ต่อ)

- สร้างโดยใช้ฟังก์ชันอันใดอันหนึ่งของ NumPy ดังต่อไปนี้
  - zeros (size) สร้างอาร์เรย์โดยให้สมาชิกทุกตัวมีค่าเริ่มแรกเป็น 0 ทั้งหมด
  - ones (size) สร้างอาร์เรย์โดยให้สมาชิกทุกตัวมีค่าเริ่มแรกเป็น 1 ทั้งหมด
  - full (size,value) สร้างอาร์เรย์โดยกำหนดจำนวนสมาชิกและละค่าเริ่มต้น (value) ตามที่ระบุ
  - arange (start. stop, step) กำหนดช่วงข้อมูลเช่นเดียวกับ range() ของ Python
  - linspace (begin, end, size) มักใช้ในเงื่อนไขที่ว่า ต้องการชุดข้อมูลระหว่าง x -y ทั้งหมด n จำนวน ทั้งนี้ต้องระบุอาร์กิวเมนต์เป็นค่าแรก ค่าสุดท้าย และจำนวนที่ต้องการ โดยช่วงข้อมูลระหว่างค่าแรกและค่าสุดท้าย จะถูกแบ่งให้มีระยะห่างเท่าๆ กัน เพื่อให้ได้จำนวนตามที่ระบุ

15. การสร้างอาร์เรย์ (ต่อ)

- random.randint (low,high,size) เมื่อต้องการเลขจำนวนเต็มที่อยู่ระหว่างค่า low และ high-1 โดย size คือจำนวนที่ต้องการ
- random.uniform (low,high,size) เมื่อต้องการเลขทศนิยมที่อยู่ระหว่างค่า low และ high 1 โดย size คือจำนวนที่ต้องการ
- random.rand (size) เมื่อต้องการอาร์เรย์เป็นเลขทศนิยมระหว่าง 0 - 1 และมีจำนวนสมาชิกตามค่า size ที่ระบุ

15. การสร้างอาร์เรย์ (ต่อ)

```
a = np.zeros (5)                    # [ 0 0 0 0 0 ]
b = np.ones (5)                     #[11111]
c = np.full(5, 8)                   # [ 8 8 8 8 8 ]
d = np.arange(1, 6)                 # [ 1 2 3 4 5 ]
e = np.arange(1, 10, 2)             # [ 1 3 5 7 9 ]
f = np.linspace(1, 10, 4)           #[1 4 7 10]
g = np.linspace(1, 10, 6)  #[1 2.8 4.6 6.4 8.2 10]
h =np.random.uniform(1,101,10)   #สุ่มเลขระหว่าง 1-100 ทั้งหมด 10 จำนวน
i = np.random.rand(10)
```

16. การเข้าถึงสมาชิกของอาร์เรย์

- การเข้าถึงสมาชิกของอาร์เรย์ก็ทำเช่นเดียวกับลิสต์ ด้วยการระบุเลขลำดับ (index) ในในวงเล็บ [ ] ซึ่งเลขลำดับอาจเป็นบวกหรือลบก็ได้ ถ้าเป็นลบจะเริ่มจากตัวสุดท้าย เช่น

```
a = np.array([7, 11, 108, 1009, 101, 555]
print(a[0])                    #7
print(a[1] + a[3])             #1020
print(a[-1])                   #555
print(a[-5])                   #11
print(*a)
for value in a: print(value)
```

16. การเข้าถึงสมาชิกของอาร์เรย์ (ต่อ)

- สามารถอ้างถึงช่วงลำดับในแบบ Index Slicing เช่นเดียวกับลิสต์คือ array [startstop:step] เช่น

```
arr = np.array([2, 7, 11, 101, 108,360,404,5555,747, 1009,2020])
print(
arr[5:],                    #[3604045555 747 1009 2020]
arr[3:9],                    #[101 108 360 404 555 747]
arr[1:10:2])                 #[7 101 360 555 1009]
```

16. การเข้าถึงสมาชิกของอาร์เรย์ (ต่อ)

- นอกจากนี้ ยังสามารถใช้วิธีการที่เรียกว่า Fancy Indexing ซึ่งเป็นการระบุเลขลำดับในแบบอาร์เรย์หรือ
- ลิสต์ เพื่ออ้างถึงหรือจัดการกับสมาชิกในลำดับเหล่านั้นไปพร้อมๆ กัน เช่น

```
arr = np.array([2, 7, 11, 101,108,360,404,555,747])
idx = [0, 2, 4, 8]              #ลำดับที่ต้องการ
a = arr [idx]
print(a)                        # [2.11 108 747]
```

17. อาร์เรย์แบบ 2 มิติ

- อาร์เรย์แบบ 2 มิติ จะคล้ายกับลิสต์แบบ 2 มิติ กล่าวคือ สมาชิกแต่ละตัวของมันก็เป็นอาร์เรย์เช่นกันหรือกล่าวได้ว่าเป็นอาร์เรย์ซ้อนอาร์เรย์นั่นเอง สามารถสร้างอาร์เรย์ซ้อนกันกี่ขั้นก็ได้ แต่ปกติจะใช้เฉพาะอาร์เรย์ที่ซ้อนกันเพียง 2 ชั้น จึงเรียกว่าเป็น อาร์เรย์ 2 มิติ ซึ่งในกรณีของ NumPy จะมีลักษณะดังนี้

```
a2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
#อาร์เรย์ย่อยลำดับ 0 => [1, 2, 3]
#อาร์เรย์ย่อยลำดับ 1 => [4, 5, 6]
#อาร์เรย์ย่อยลำดับ 2 => [7, 8, 9]
```

17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- ในรูปแบบของ NumPy มักจะนำอาร์เรย์ย่อยแต่ละอันมาเขียนเรียนเรียงในแนวตั้ง ส่วนสมาชิกของอาร์เรย์ย่อยก็เขียนเรียงในแนวนอนตามปกติ แล้วเรียกอาร์เรย์แต่ละอันที่วางซ้อนกันว่า แถว และเรียกสมาชิกของอาร์เรย์ย่อยในแนวหรือลำดับเดียวกันว่า คอลัมน์ ซึ่งก็คล้ายกับตารางนั่นเอง ดังภาพ
- อย่างไรก็ตาม ข้อกำหนดที่สำคัญคือ อาร์เรย์ย่อยแต่ละอันต้องมี จำนวนสมาชิกเท่ากันทั้งหมด หรือถ้าพิจารณาตามหลักการในดังภาพละแถวต้องมีจำนวนคอลัมน์เท่ากันนั้นเอง ซึ่งหากไม่เท่ากันจะกลายเป็นอาร์เรย์ของลิสต์ ไม่ใช่อาร์เรย์ของอาร์เรย์ และอาจเกิดปัญหาเมื่อจะดำเนินกับข้อมูลในอาร์เรย์ดังกล่าว



17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- ในลำดับต่อไป หากระบุว่าเป็นอาร์เรย์ขนาด 3 4 หรือ (3, 4) ก็หมายถึง อาร์เรย์ที่มีขนาด 4 คอลัมน์ เช่น

```
[[10, 20, 30, 40],
[50, 60, 70, 80],
[90, 100, 11, 12]]
```

17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- ในการสร้างอาร์เรย์ 2 มิติ สามารถนำบางฟังก์ชันจากอาร์เรย์แบบมิติเดียวมาใช้ได้ แต่กรณีนี้นี้จะต้องระบุจำนวนแถวและคอลัมน์ในแบบลิสต์หรือทูเพิลลงไปด้วย ดังนี้ (rows คือจำนวนแถว และ cols คือจำนวนคอลัมน์)

zeros([rows.cols]) เช่น np.zeros([3, 4]) สร้างอาร์เรย์ขนาด 3 4

ones([rows, cols]) เช่น np.ones((5. 5]) สร้างอาร์เรย์ขนาด 5x5

full([rows, cols], value)

random.randint(low, high, [rows, cols])

random.uniform(low, high, [rows,cols])

random.rand(rows, cols)

17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- หากต้องการอ้างถึงสมาชิกของอาร์เรย์ 2 มิติ ต้องระบุแถวและคอลัมน์ในวงเล็บ [ ] ดังรูปแบบต่อไปนี้

array[ลำดับแถว, ลำดับคอลัมน์]

- ลักษณะการอ้างถึงสมาชิกดังกล่าว จะแตกต่างจากลิสต์แบบ 2 มิติ ที่ต้องใช้วงเล็บ 2 อัน เช่น [1[4] แต่อาร์เรย์ของ NumPy จะใช้วงเล็บ [ ] เพียงอันเดียว เช่น [2, 5] ดังแนวทางดังต่อไปนี้

```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8,
print(a, a[0, 0], a[1, 3], a[2, 2], sep =‘ \n\n')
```

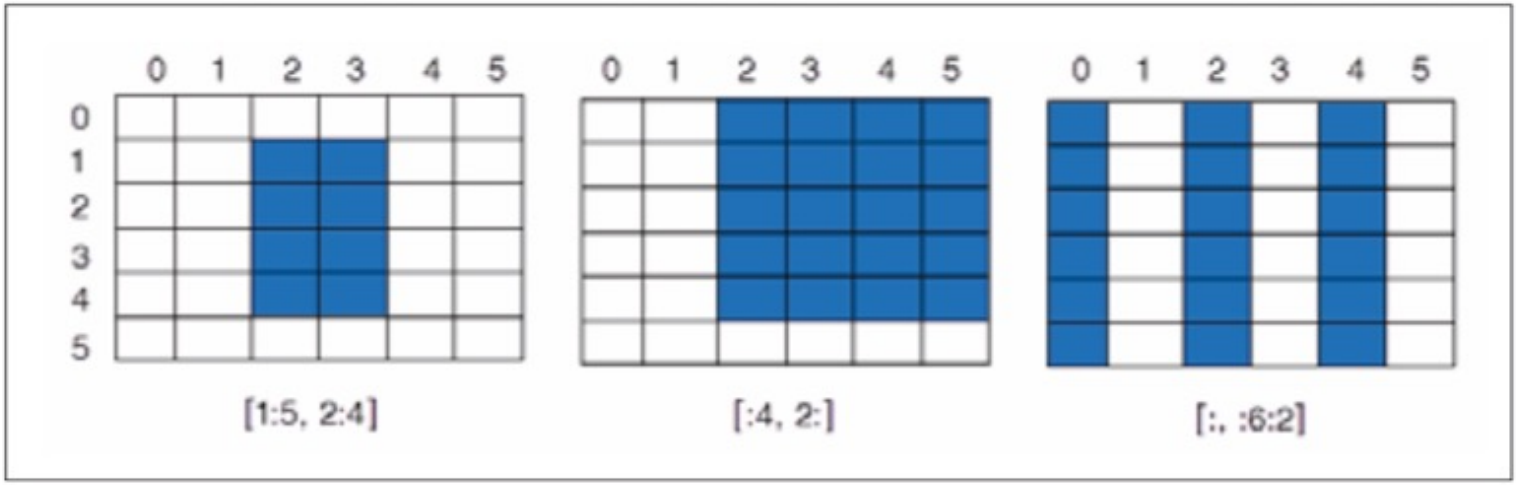
17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- หากต้องการอ้างถึงสมาชิกของอาร์เรย์ที่เป็นลำดับต่อเนื่องกัน ก็สามารถใช้วิธี Index Slie เดียวกัน โดยการระบุช่วงลำดับในรูปแบบ

array[start_row:stop_row:step, start_column:stop_column:step]

17. อาร์เรย์แบบ 2 มิติ (ต่อ)

- กลุ่มแรกหมายถึงลำดับแถว ส่วนกลุ่มหลังหมายถึงลำดับคอลัมน์ เช่น
- [1:5, 2:4] เลือกลำดับแถวที่ 1-4 และคอลัมน์ที่ 2 2-3
- [:4, 2:] เลือกแถวที่ 0-3 และคอลัมน์ที่ 2 เป็นต้นไป
- [:, :6:2] เลือกทุกแถว และคอลัมน์ที่ 0-5 โดยมีระยะห่างระหว่างคอลัมน์เท่ากับ 2 หรือลำดับที่ 0,2, 4 นั่นเอง



18. การคำนวณอาร์เรย์ด้วยโอเปอเรเตอร์

- สามารถนำอาร์เรย์ของ NumPy มาดำเนินการด้วยเครื่องหมาย หรือโอเปอเรเตอร์ทางคณิตศาสตร์ เช่น +, - It / หรือ * รวมถึงไอเปอเรเตอร์สำหรับการเปรียบเทียบ เช่น >, < หรือ = เป็นต้น โดยมีแนวทางดัง ต่อไปนี้ (กรณีอาร์เรย์ 2 มิติก็ใช้หลักการเดียวกัน)
- หากนำอาร์เรย์ไปดำเนินการกับตัวเลข เช่น array + 10 จะเป็นการกระทำกับสมาชิกทุกตัว ในลักษณะดังนี้

```
arr = np.array([7, 11, 108, 1009, 101, 555])
arr2 = arr + 10 #[17 21 118 1019 111 565]
```

18. การคำนวณอาร์เรย์ด้วยโอเปอเรเตอร์ (ต่อ)

- หากดำเนินการระหว่างอาร์เรย์กับอาร์เรย์ เช่น array + array จะเป็นการกระทำระหว่างสมาชิกของอาร์เรย์ในลำดับที่ตรงกัน แต่อาร์เรย์ทั้งสองอัน ต้องมีจำนวนสมาชิกเท่ากัน ดังแนวทางต่อไปนี้

```
a = np.array([2, 7, 11, 101])
b = np.array([10, 8, 9, 99])
a + b      #[12 15 20 200]
```

18. การคำนวณอาร์เรย์ด้วยโอเปอเรเตอร์ (ต่อ)

- หากต้องการคัดกรองเอาเฉพาะข้อมูลที่ตรงตามเงื่อนไข ก็สามารถใช้โอเปอเรเตอร์สำหรับ การเปรียบเทียบ เช่น >,>=, <, <= หรือ != เป็นต้น ดังแนวทางต่อไปนี้

```
arr = np.random.randint(10, 101, 10)
a = arr[arr >= 50]            #สมาชิกของ arr ที่มีค่าตั้งแต่ 50 ขึ้นไป
b = arr[arr % 2 == 0]         #สมาชิกของ arr ที่หาร 2 ลงตัว
c = arr[(arr >= 20) & (arr <= 60)] #สมาชิกของ arr ที่อยู่ระหว่าง 20 - 60
d = arr[(arr != 33) | (arr != 51) | (arr != 89)]
```

19. ฟังก์ชันที่น่าสนใจของ Numpy

- ไลบรารี NumPy มีฟังก์ชันเป็นจำนวนมาก สำหรับในที่นี่จะนำมากล่าวเฉพาะอันที่น่าสนใจ ซึ่งวัตถุประสงค์โดยสังเขปของแต่ละฟังก์ชันมีดังต่อไปนี้

- sum() หาผลรวมของสมาชิกอาร์เรย์
- min(), max() หาค่าต่ำสุดและสูงสุดของสมาชิกตามลำดับ
- mean() หาค่าเฉลี่ยของสมาชิก
- unique() เลือกสมาชิกที่มีค่าไม่ซ้ำกัน
- reshape() เปลี่ยนขนาดของอาร์เรย์

19. ฟังก์ชันที่น่าสนใจของ Numpy (ต่อ)

- ฟังก์ชันสำคัญที่ต่อไปต้องใช้งานอยู่บ่อย ๆ ก็คือ reshape() เป็นการเปลี่ยนขนาดของอาร์เรย์ เช่น 3x8 x 8 หรือ x 12 เป็นต้น หรืออาจแก้ไขจากอาร์เรย์มิติเดียวให้เป็น 2 มิติก็ได้ แต่อาร์เรย์ใหม่ต้องมีจำนวนสมาชิกเท่ากับอาร์เรย์เดิม

```
a1 = np.arange(1, 11)                #[12345678910]
ar = a1.reshape(2, 5)                #[[12345]4 5], [6 7 8 9 10]]
#ฟังก์ชัน arange(1, 11) จะได้อาร์เรย์มิติเดียว ที่มีจำนวนสมาชิก 10 ตัว
#แล้วใช้ reshape(2, 5) เพื่อเปลี่ยนเป็นอาร์เรย์ 2 มิติขนาด 2 x 5 หรือเขียนเป็น
#ar = np.arange(1, 11).reshape([2, 5])
2 = np.random.randint(1, 10, (2, 6)) #อาร์เรย์ขนาด
b = a2.reshape(3, 4)                 #อาร์เรย์ขนาด 3x4
```

19. ฟังก์ชันที่น่าสนใจของ Numpy (ต่อ)

- ยังมีลักษณะที่เพิ่มเติมอีกอย่างหนึ่งก็คือ การระบุจำนวนแถวหรือคอลัมน์ เพียงอย่างใดอย่างหนึ่ง ให้เป็น -1 เช่น array.reshape(-1,3) หรือ array.reshape (2. -1) ซึ่งมีความหมายดังนี้
  - array.reshape (2,-1) หมายความว่า ต้องการเปลี่ยนขนาดให้เป็นอาร์เรย์แบบ 2 แถวจำนวนคอลัมน์จึงระบุค่าเป็น -1 ลงไปแทน เพื่อให้ NumPy คำนวณจำนวนคอลัมน์ให้เอง เช่น ถ้าอาร์เรย์เดิมเป็นขนาด (3. 4) การกำหนด array.reshape (2,1) จึงเทียบเท่ากับ array.reshape (2, 6)
  - array.reshape(-1, 3) หมายความว่า ต้องการเปลี่ยนขนาดให้เป็นอาร์เรย์แบบ 3 คอลัมน์แต่ไม่ทราบจำนวนแถวจึงระบุค่าเป็น -1 ลงไปแทน เพื่อให้ NumPy คำนวณจำนวนแถวให้เอง เช่น ถ้าอาร์เรย์เดิมเป็นขนาด (6. 2) การกำหนด array.reshape(-1. 3) จึงเทียบเท่ากับ arrav.reshape (4. 3)

19. ฟังก์ชันที่น่าสนใจของ Numpy (ต่อ)

```
arr = np.arange(1, 13)
a = arr.reshape(2, -1)          #แปลงเป็นขนาด [2, 6]
b = arr.reshape(4, -1)          #แปลงเป็นขนาด [4, 3]
c = arr.reshape(-1, 3)          #แปลงเป็นขนาด [4, 3]
d = arr.reshape(-1, 6)          #แปลงเป็นขนาด [2, 6]
```

20. ไลบรารี Pandas และการใช้ DataFrame

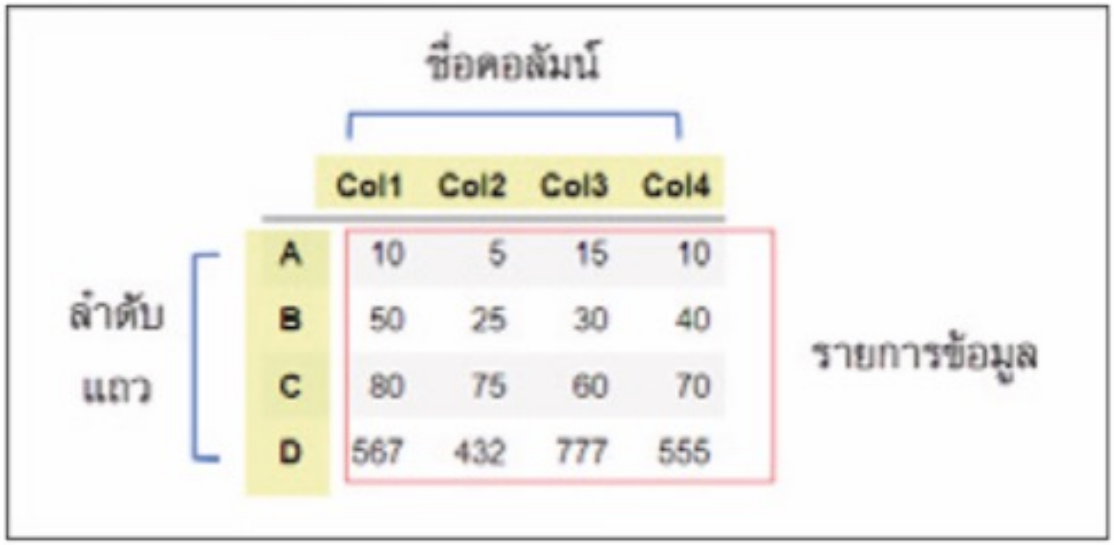
- Pandas มาจากคำว่า Panel Data มีวัตถุประสงค์หลักคือ เพื่อกำหนดและวิเคราะห์ข้อมูลในรูปแบบของตาราง และถือเป็นไลบรารีหลักของการใช้งานเกี่ยวกับ Data Science แต่เนื่องจาก Pandas ไม่ใช่องค์ประกอบพื้นฐานของภาษา Python ซึ่งตามปกติก็ต้องติดตั้งเพิ่มเติมลงไปเอง อย่างไรก็ตาม หากติดตั้งด้วยชุดแพ็กเกจ Anaconda ก็จะมีไลบรารี Pandas พร้อมให้ใช้งานได้ทันที โดยหลังจากเปิดเข้าสู่ Jupyter ให้ระบุคำสั่ง import เพื่ออ้างอิงไลบรารี Pandas ดังนี้ pd คือชื่อ (allas) ที่ใช้อ้างอิงแทนโมดูล pandas ซึ่งนิยมใช้ชื่อนี้กันเป็นส่วนใหญ่
- ไลบรารี Pandas มีออบเจ็กต์ในการกำหนดข้อมูล 2 อย่างคือ Series และ DataFrame แต่ในที่นี้จะกล่าวถึงเฉพาะ DataFrame เท่านั้น เพราะเป็นออบเจ็กต์หลักที่ต้องนำไปใช้ร่วมกับ Machine Learning โดยลักษณะการใช้งานที่สำคัญมีดังนี้

```
1  import pandas as pd
2
3  print(pd.__version__)
```

```
1.0.5
```

21. การสร้างและกำหนดข้อมูลให้กับ DataFrame

- การสร้างออบเจ็กต์ DataFrame มีรูปแบบการกำหนดอาร์กิวเมนต์ที่สำคัญในลักษณะดังนี้



pd.DataFrame(data, index, columns)

21. การสร้างและกำหนดข้อมูลให้กับ DataFrame (ต่อ)

- data คือข้อมูลที่จะจัดเก็บใน DataFrame ซึ่งอาจอยู่ในรูปแบบ ดิกชันนารี ลิสต์ ทูเพิล หรืออาร์เรย์ของ NumPy ก็ได้ แต่ต้องเป็นแบบ 2 มิติ
- index คือลำดับแถว อาจกำหนดเป็นตัวเลขหรือสตริงก็ได้ เช่น 'A', 'B', 'C'.... หรือ ', 'Mon',Tue', ... เป็นต้น หากไม่กำหนด ลำดับแถวจะเป็นตัวเลข 0, 1
- columns คือชื่อของแต่ละคอลัมน์ อาจกำหนดในรูปแบบลิสต์ หรือทูเพิล เรียงจากคอลัมน์เป็นต้นไป หากไม่ระบุ ชื่อคอลัมน์จะเป็นตัวเลข 0, 1. 2. 3...

- ส่วนการกำหนดข้อมูลให้กับ DataFrame มีให้เลือกหลายวิธี ดังต่อไปนี้
- หากมีข้อมูลอยู่แล้วและสามารถระบุค่าที่แน่นอนลงไปได้ ก็อาจกำหนดข้อมูลด้วยลิสต์หรือทูเพิลในแบบ 2 มิติให้กับ DataFrame ดังแนวทางต่อไปนี้

21. การสร้างและกำหนดข้อมูลให้กับ DataFrame (ต่อ)

```
import pandas as pd

data = [[10, 50, 80, 567],
        [5, 25, 75, 432],
        [15, 30, 60, 777],
        [10, 40, 70, 555]]

df = pd.DataFrame(data,
                  index=list('ABCD'),
                  columns=['One', 'Two', 'Three', 'Four'])

df
```

	One	Two	Three	Four
A	10	50	80	567
B	5	25	75	432
C	15	30	60	777
D	10	40	70	555



21. การสร้างและกำหนดข้อมูลให้กับ DataFrame (ต่อ)

- นอกจากนี้ อาจใช้รูปแบบของดิกชันนารี หรือ Key/Value โดยค่า Key จะเป็นชื่อคอลัมน์จะเป็นข้อมูลในแต่ละคอลัมน์ ซึ่งส่วน Value อาจอยู่ในรูปแบบของลิสต์หรืออาร์เรย์ของ Numpy ทูเพิลก็ได้ ในลักษณะดังนี้

```
data = [ 'column1 : [row1, row2, row3, ...],
'column2':[rowi,row2, row3, ....],
'column3' : [row1, row2, row3..],
…
}
```

21. การสร้างและกำหนดข้อมูลให้กับ DataFrame (ต่อ)

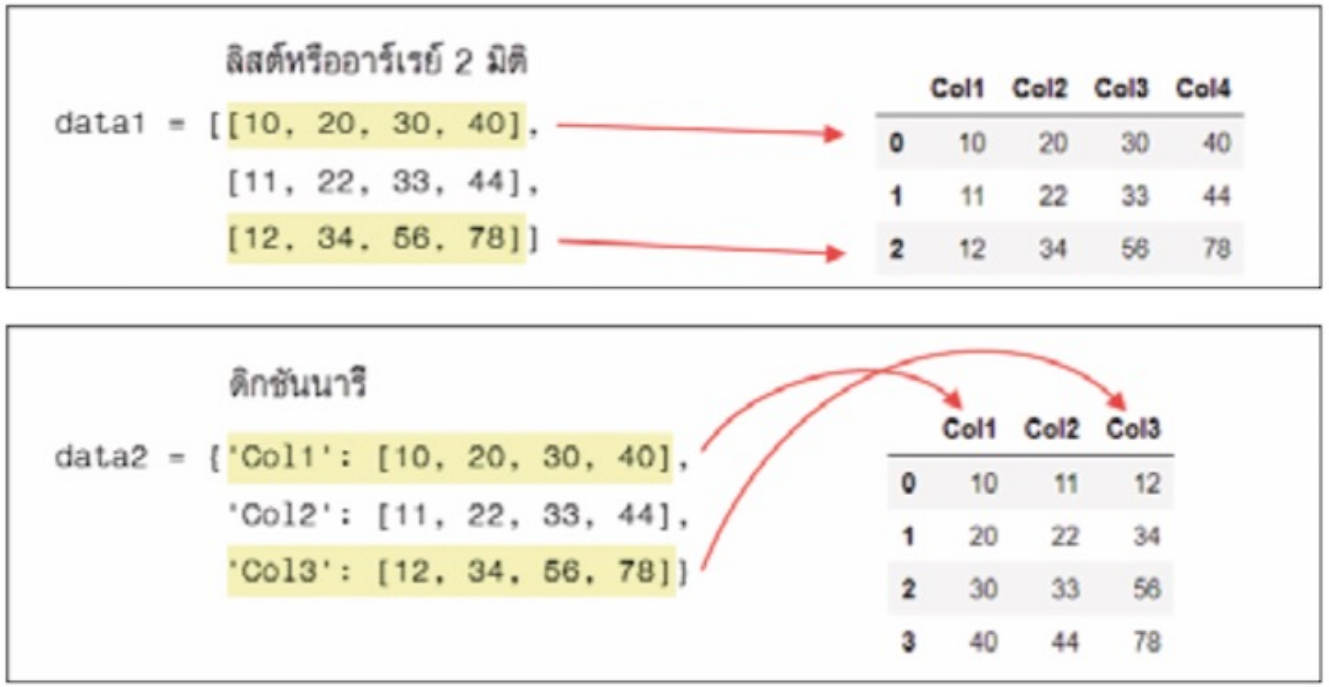
- ข้อดีของวิธีนี้คือ ชื่อ Key ของแต่ละรายการจะถูกนำไปตั้งเป็นชื่อคลัมน์ ส่วน Value จะเป็นข้อมูลในแต่ละคอลัมน์ แต่ต้องมีจำนวนเท่ากันทุกคอลัมน์ เช่น...

	Col1	Col2	Col3
p1	110	210	310
p2	111	211	311
p3	112	212	312
p4	113	213	313

```
import pandas as pd
data = ( 'Col1': [110, 111, 112, 113],
'Co12': [210,211,21212, 213],
'Co13': [310, 311, 312, 313] )
df = pd.DataFrame(data, index=['p1', 'p2','p3','p4'p4'])
```

21. การสร้างและกำหนดข้อมูลให้กับ DataFrame (ต่อ)

- การกำหนดข้อมูลในแบบดิกชันนารีให้กับ DataFrame ตามที่ได้กล่าวมา จะเห็นได้ว่า สมาชิกแบบ Key/ Value ของดิกชันนารีที่จัดเรียงในแนวนอน จะกลายเป็นข้อมูลในแต่ละคอลัมน์ของ DataFrame ที่จัดเรียงในแนวตั้ง ซึ่งจะแตกต่างจากกรณีที่กำหนดค่าด้วยลิสต์หรืออาร์เรย์แบบ 2 มิติ ที่รายการในแนวนอนก็จะเป็นข้อมูลในแต่ละแถว ดังการเปรียบเทียบในภาพต่อไปนี้



22. การเลือกข้อมูลใน DataFrame

- ไม่จำเป็นต้องนำข้อมูลที่มีใน DataFrame ไปใช้งานทั้งหมด แต่อาจคัดเลือกเอาเพียงบางแถว และ/หรือ บางคอลัมน์ ซึ่งวิธีการเลือกข้อมูลจาก DataFrarne มีหลากหลายรูปแบบ แต่ในที่นี้จะกล่าวถึงเพียงลักษณะที่มักได้ใช้งานร่วมกับ Machine Learning อยู่บ่อยๆ เท่านั้น
- หากต้องการอ้างถึงข้อมูลทั้งคอลัมน์ หรือถ้าต้องการอ้างถึงข้อมูลในเซลล์ใดเซลล์หนึ่ง ให้ระบุทั้งแถวและคอลัมน์ดังรูปแบบต่อไปนี้

```
df[ลำดับคอลัมน์]
df[ลำดับคอลัมน์] [ลำดับแถว]

a = df[0]#คอลัมน์ลำดับที่ 0 ทั้งคอลัมน์ (ทุกแถว)
b = df[2]          #คอลัมน์ลำดับที่ 2 ทั้งคอลัมน์ (ทุกแถว)
c = df[0][0]       #คอลัมน์ลำดับที่ 0 แถวลำดับที่ 0
d = df[1][2])      #คอลัมน์ลำดับที่ 1 แถวลำดับที่ 2
```

22. การเลือกข้อมูลใน DataFrame (ต่อ)

- ถ้าตอนสร้าง DataFrame กำหนดชื่อคอลัมน์ให้กับมัน ก็สามารถอ้างอิงผ่านชื่อคอลัมนี้ใช้รูปแบบ

df ['ชื่อคอลัมน์']
หรือ df.ชื่อคอลัมน์

df ['ชื่อคอลัมน์']] [แถว]
หรือ df.ชื่อคอลัมน์[แถว]

22. การเลือกข้อมูลใน DataFrame (ต่อ)

```
import pandas as pd
data = [ 'price': [200, 300, 250, 50
'sales': [0, 30, 25, 50],
'stock': [8, 0, 5, 1] ]
df = pd.DataFrame(data, index=['p1', 'p2','p3','p4'])
display(df['price’],          #คอลัมน์ price
df[ ' sales' ] [2] ,          #คอลัมน์ sales แถวลำดับที่ 2
df.stock,                     #คอลัมน์ stock
df.stock[1],                  #คอลัมน์ stock แถวลำดับที่ 1
df.stock['p4’])               #คอลัมน์ stock แถว p4
```

22. การเลือกข้อมูลใน DataFrame (ต่อ)

- กรณีที่ต้องการเลือกหลายๆ คอลัมน์แบบเจาะจงก็อาจใช้รูปแบบดังนี้ (start/stop/step ใช้ช่วงแถวด้วยหลักการเดียวกับ NumPy)

```
df[['coli', 'col2', 'col3', ...]]
df[start_row : stop_row : step][['coli', 'co12', 'co13', ...]]
```

หรืออาจจะใช้ iloc ร่วมกับ Index Slicing ดังรูปแบบต่อไปนี้

```
df.iloc[start_row : stop_row : step, start_column : stop_column : step]
หรือ df.iloc[start_row : stop_row: step,[c1, c2,c3,....]]
#C1, c2, ..คือเลขลำดับคอลัมน์
```

22. การเลือกข้อมูลใน DataFrame (ต่อ)

```
import numpy as np
import pandas as pd
ata = [ 'One': np.random.randint(100, 1000,
'Two': np.random.uniform(1, 10,10),
'Three': np.random.rand(10),
'Four': np.random.randint(1, 100, 10),
'Five': np.random.uniform(-10, 10, 10) )
df = pd.DataFrame(data)
display(df[['One', 'Three', 'Five']],
df[:][['Two', 'Three', 'Four']],
df.iloc[2:5, 3:5],   #แถวลำดับที่ 2 - 4 และคอลัมน์ลำดับที่ 3 - 4
df.iloc[::, [0, 2, 4]])          #ทุกแถว คลล์มน้ำลักตั้กพันที่ 0, 2 และ 4
```

22. การเลือกข้อมูลใน DataFrame (ต่อ)

- นอกจากนี้ สามารถใช้โอเปอเรเตอร์การเปรียบเทียบ เช่น >, <, !=, == หรืออื่นๆ ที่ได้ผลลอกมาเป็น True หรือ False เพื่อเลือกหรือคัดกรองเอาเฉพาะข้อมูลที่ตรงตามเงื่อนไข ด้วยหลักการเดียวกับ NumPy เช่น

```
import pandas as pd
data = ( 'price': [200,300, 250,500],
'sales': [0, 30, 25, 50],
'stock': [8, 0, 5, 1] ]
df = pd.DataFrame(data)
display(df[df['price'] >= 300],                    #ราคา 300+
[(df.price >= 200) & (df.price <= 300)], #ราคา 20
df[df['stock'] == 0],
df[['price', 'stock']]][df.stock > 0])  #stock มากกว่า 0
```

23. ฟังก์ชันที่น่าสนใจของ DataFrame

- DataFrame มีฟังก์ชันให้เลือกใช้เป็นจำนวนมาก แต่ฟังก์ชันที่น่าสนใจซึ่งอาจได้นำไปใช้ร่วมกับ Machine Learning ในบางโอกาส ดังนี้

count()	นับจำนวนรายการข้อมูลที่ไม่ใช่ NaN ในแต่ละคอลัมน์
sum()	หาผลรวมของข้อมูลในแต่ละคอลัมน์
min() / max()	หาค่าที่น้อยที่สุด มากที่สุด ในแต่ละคอลัมน์
mean()	หาค่าเฉลี่ยของข้อมูลในแต่ละคอลัมน์
describe()	แสดงค่าทางสถิติที่สำคัญ
info()	แสดงรายละเอียดเกี่ยวกับ DataFrame
head(n)	แสดงรายการ n แถวแรก (ถ้าไม่ระบุจำนวน จะแสดง 5 แถวแรก)
tail(n)	แสดงรายการ n แถวสุดท้าย (ถ้าไม่ระบุจำนวน จะแสดง 5 แถวสุดท้าย)
rename()	สำหรับแก้ไขชื่อคอลัมน์ โดยแมปชื่อที่ตรงกันในแบบดิกชันนารี
replace()	แทนที่ข้อมูลเดิมด้วยข้อมูลใหม่ในคอลัมน์ที่ระบุ โดยใช้รูปแบบ replace({ข้อมูลเดิม1: ค่าใหม่1, ข้อมูลเดิม2: ค่าใหม่2, ...})
drop(columns= ['col1', col2', ...])	ลบคอลัมน์ที่ระบุออกจาก DataFrame

23. ฟังก์ชันที่น่าสนใจของ DataFrame (ต่อ)

```
import numpy as np
import pandas as pd
data = ( 'One': np.random.randint(100, 1000, 10),
'Two': np.random.uniform(1, 10,10),
'Three': np.random.rand(10),
'Four': np.random.randint(1, 100, 10),
'Five': np.random.uniform(-10, 10,10) )
df = pd.DataFrame(data)
display (
df.sum(),  #หาผลรวมแต่ละคอลัมน์
df.sum() ['One'],       #หาผลรวมของคอลัมน์ 'One'
df.max(),
df.max() [ 'One'],
df.describe()
)
```

23. ฟังก์ชันที่น่าสนใจของ DataFrame (ต่อ)

```
df.info()  #info() ไม่ต้องใช้ร่วมกับ print'() หรือ display()
df.rename(columns=('One':'Colt', 'Two': 'Co12’))
#เปลี่ยนชื่อคอลัมน์ 'One' เป็น 'Co11' และคอลัมน์ 'Two' เป็น 'Co12'
df.replace((10: 11, 20: 22, 30: 33), inplace=True)
#หมายถึงให้เปลี่ยนค่า 10 เป็น 11 , เปลี่ยน 20 เป็น 22, .
lace=True หมายถึงให้บันทึกผลการเปลี่ยนแปลงลงในออบเจ็กต์นี้
#มิฉะนั้น ต้องสร้างตัวแปรมารับค่า เช่น df2 = df.replace(...)
df.drop(columns=['One', 'Four'], inplace-True)
#ลบคอลัมน์ 'One' และ 'Four' ออกจาก DataFrame
```

23. ฟังก์ชันที่น่าสนใจของ DataFrame (ต่อ)

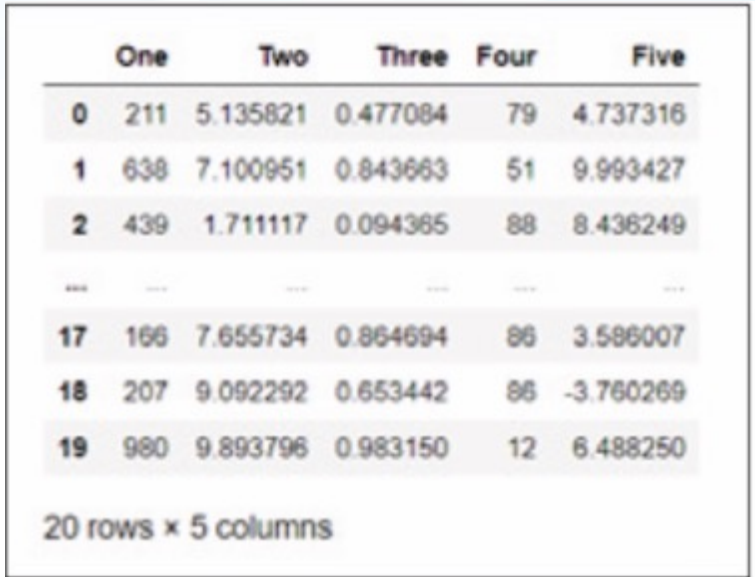
- หากต้องการแสดงผลทั้งช่วงแถวแรกและช่วงแถวสุดท้ายร่วมกัน อาจใช้วิธีการกำหนดออปชันในรูปแบบดังนี้

```
with pd.option_context(('display.max_rows', จำนวนแถวที่จะแสดง): display(df)
```

- จำนวนที่แสดงจะถูกแบ่งครึ่งระหว่างช่วงแถวแรกและช่วงแถวสุดท้าย เช่น หากกำหนดเป็นเลข 10 ก็จะแสดง 5 แถวแรกและ 5 แถวสุดท้าย เป็นต้น ดังแนวทางในโค้ดต่อไปนี้

23. ฟังก์ชันที่น่าสนใจของ DataFrame (ต่อ)

```
import numpy as np
import pandas as pd
data = ( 'One': np.random.randint
(100, 1000, 20),
'Two': np.random.uniform(1, 10,20),
'Three': np.random.rand(20),
'Four': np.random.randint(1, 100, 20),
'Five : np.random.uniform(-10, 10,20) )
df = pd.DataFrame(data)
with pd.option_context( display.max_rows', 6): display(df)
```



	One	Two	Three	Four	Five
0	211	5.135821	0.477084	79	4.737316
1	638	7.100951	0.843663	51	9.993427
2	439	1.711117	0.094365	88	8.436249
...	...	...	...	...	...
17	166	7.655734	0.864694	86	3.586007
18	207	9.092292	0.653442	86	-3.760269
19	980	9.893796	0.983150	12	6.488250

20 rows × 5 columns

24. การวาดกราฟด้วยไลบรารี Matplotlib

- Matplotib ใช้สำหรับการวิเคราะห์ความสัมพันธ์ระหว่างข้อมูลในรูปแบบของกราฟชนิดต่าง ๆ เช่น กราฟเส้น กราฟแท่ง กราฟวงกลม เป็นต้น อย่างไรก็ตาม ไลบรารี Matplotib ไม่ใช่องค์ประกอบพื้นฐานของภาษา Python ซึ่งตามปกติหากจะใช้งานก็ต้องติดตั้งเพิ่มเข้ามาเอง แต่ถ้าติดตั้งด้วยชุดแพ็กเกจ Anaconda ก็จะมีไลบรารี Matplotib รวมอยู่ด้วย ดังนั้น จึงสามารถใช้งานร่วมกับ Jupyter ได้ทันที โดยสั่งอิมพอร์ตโมดูล matplotlib.pyplot ดังแนวทางต่อไปนี้

```
1       import matplotlib.pyplot as plt
2
3       print(plt)
<module 'matplotlib.pyplot' from 'C:1\Users)(Adm
in\\Anaconda3\\1ib\|site-packages\(matplotlibllp
yplot.py'>
```

25. การวาดกราฟเส้น

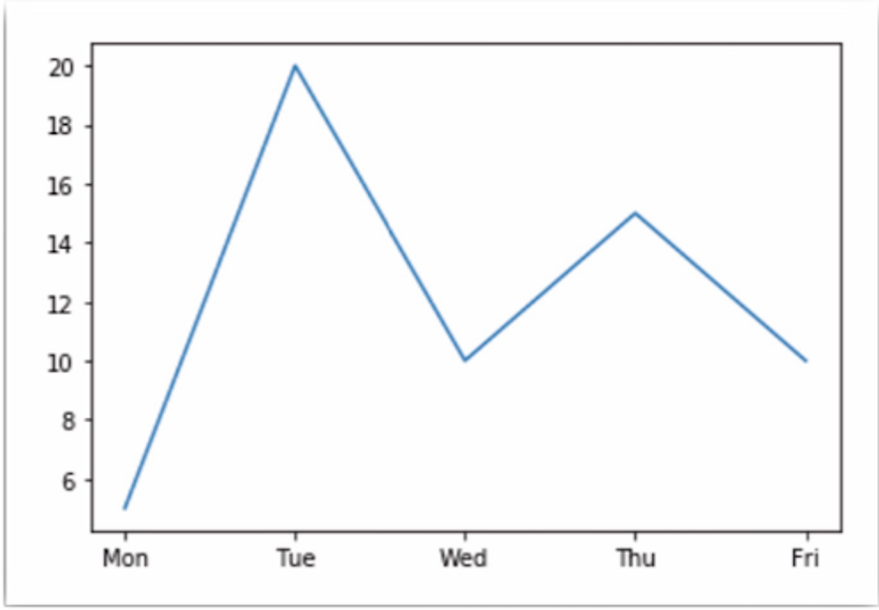
- กราฟเส้น (Linear Plot) เป็นการลากเส้นเชื่อมโยงระหว่างจุดที่แสดงความสัมพันธ์ของข้อมูลในแนวแกน X และแกน Y โดยมีขั้นตอนในเบื้องต้นดังนี้

1. กำหนดข้อมูลในแนวแกน X ซึ่งอาจเป็นแบบลิสต์ ทูเพิล หรืออาร์เรย์ (NumPy)
2. กำหนดข้อมูลในแนวแกน Y ซึ่งอาจเป็นแบบลิสต์ ทูเพิล หรืออาร์เรย์ (NumPy)
3. สร้างกราฟเส้นด้วยฟังก์ชัน plot() ของไลบรารี Matplottib โดยนำข้อมูลของแนวแกน X และ Y มากำหนดให้กับมัน
4. สั่งแสดงกราฟด้วยฟังก์ชัน show()

25. การวาดกราฟเส้น (ต่อ)

- แนวทางการวาดกราฟเส้นในเบื้องต้นจะมีลักษณะดังนี้

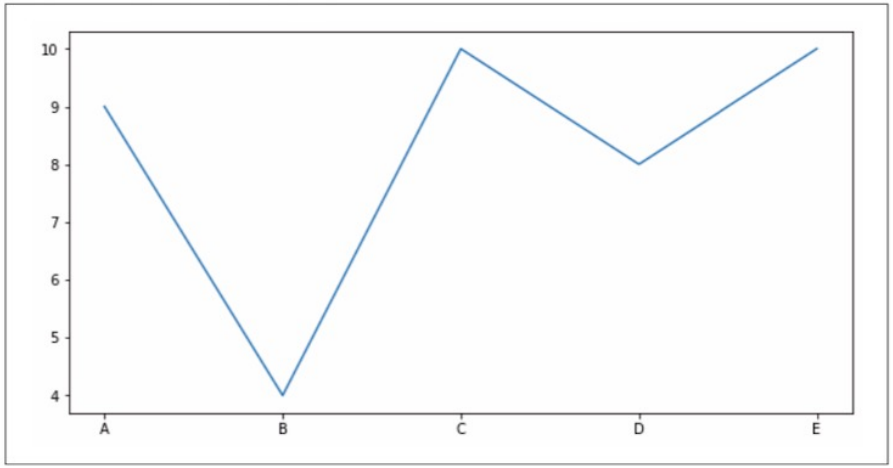
```
mport matplotlib.pyplot as plt
y = [5, 20, 10, 15, 10]
x = ['Mon', 'Tue', 'Wed',
'Thu', 'Fri']
plt.plot(x, y)
plt.show()
```



25. การวาดกราฟเส้น (ต่อ)

- หากขนาดปกติของกราฟเล็กหรือใหญ่จนเกินไป สามารถปรับขนาดความกว้างและสูงด้วยพร็อปเพอร์ตี้ rcParams หรือใช้เมธอด figure(figsize=) โดยกำหนดขนาดในแบบบทูเพิล (หน่วยเป็นนิ้ว) เช่น

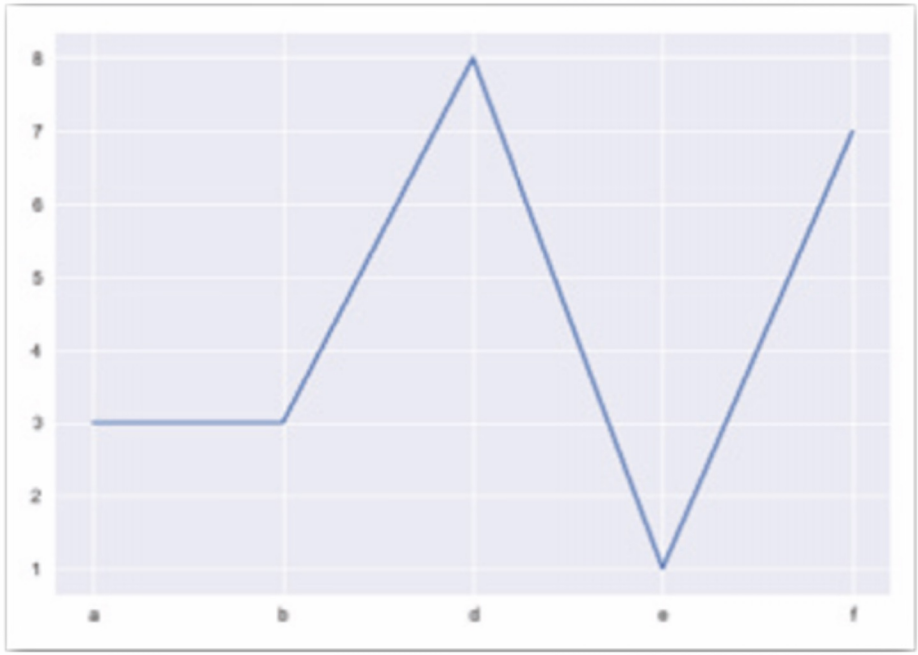
```
import matplotlib.pyplot as plt
y = np.random.randint(1, 11,5)
x = list('ABCDE')
plt.rcParams['figure.figsize'] = (10, 5) #
#หรืออีกแบบคือ
#plt.figure(figsize=(10, 5))
plt.plot(x, y)
plt.show()
```



25. การวาดกราฟเส้น (ต่อ)

- Matplotlib มีฟังก์ชัน style. use() สำหรับกำหนดรูปแบบของกราฟ ดังแนวทางต่อไปนี้

```
#ใช้สไตล์ชื่อ seaborn
plt.style.use ( seaborn')
y = np.random.randint(1, 11,5)
x = list('abdef')
plt.plot(x, y)
plt.show()
```



26. การกำหนดสีของกราฟ

- มีองค์ประกอบบางอย่างของกราฟที่สามารถกำหนดสีสันให้กับมันได้ ทั้งนี้ก็เพื่อแยกความแตกต่างของข้อมูลแต่ละอย่างให้ชัดเจนยิ่งขึ้น ดังนี้
- รูปแบบการกำหนดสีที่ Matplotib รองรับคือ
  - Color Name เช่น color='red' หรือ color='orange' เป็นต้น
  - Single Character คือการกำหนดด้วยอักขระตัวเดียว ซึ่งมีเพียง 8 สี เท่านั้นคือ

อักขระ	ชื่อสี	แนวทางการใช้งาน
r	red	color='r
g	green	color='g'
b	blue	color='b'
c	cyan	c='c'
m	magenta	c='m'
y	yellow	c='y'
w	white	c='w'
k	black	c='k'

26. การกำหนดสีของกราฟ (ต่อ)

- #Hex โดยกำหนดค่าแบบสตริงให้ขึ้นต้นด้วย # แล้วตามอักขระ a-f หรือ 0-9 จำนวน 6 ตัว เช่น color='#af09c7'
- RGB กำหนดค่าระหว่าง 0-1 สำหรับระดับสีแดง เขียว และน้ำเงิน ในแบบ (r, g, b) หรือ (a, r, g, b) เมื่อ a คือค่า alpha เช่น

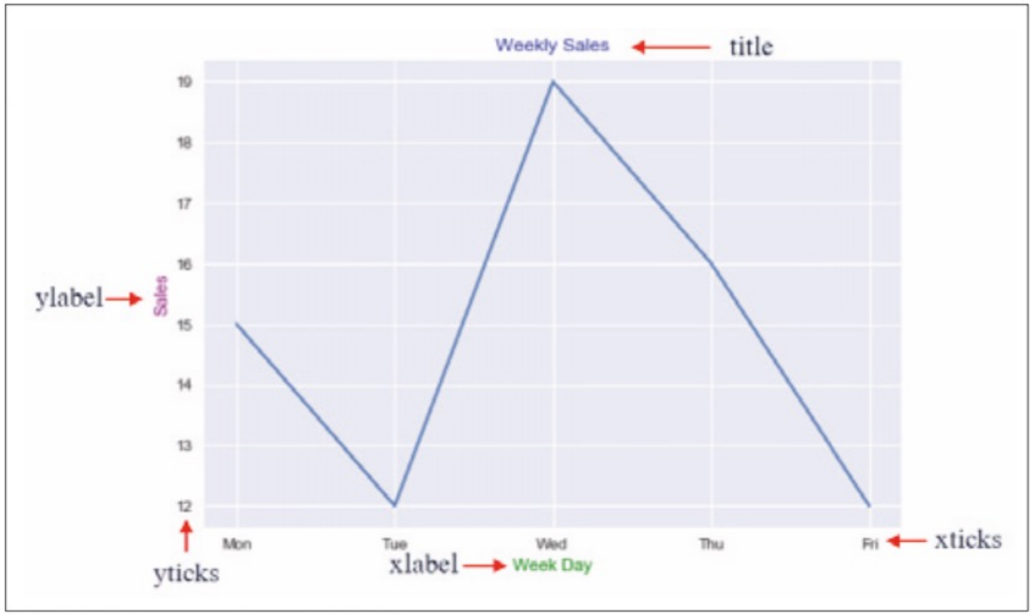
  color=(0.3. 0.4, 0.9)

  color=(0.4, 0.5, 0.75, 0.6)

- ในบางกรณี สามารถลดระดับความเข้ม (ทึบ) ของสี หรือ opaque เพื่อให้พิจารณาองค์ประกอบส่วนอื่นๆ ให้ง่ายขึ้น เช่น สีของเส้นกริดอาจให้เป็นสีจาง เพื่อไม่ให้กลมกลืนกับสีของเส้นกราฟ ซึ่งวิธีการก็คือ ให้กำหนดด้วยอาร์กิวเมนต์ alpha เป็นตัวเลขระหว่าง 0-1 เช่น alpha=0.3 โดยตัวเลขมาก หมายถึงระดับความทึบก็จะยิ่งมาก เช่น 0.5 มีความทึบมากกว่า 0.1 เป็นต้น

27. การกำหนด Label, Title และ Tick

- เพื่อให้ทราบว่ากราฟ มีข้อมูลในแต่ละแนวแกนนั้นหมายถึงสิ่งใด อาจเขียนข้อความ (Label) กำกับเอาไว้บนแกนทั้งสอง รวมถึงชื่อของกราฟ (Tie) อีกด้วย ส่วน Tick ใช้ในการแบ่งสเกลของแต่ละแกน ซึ่งอาจเป็นตัวเลขหรือข้อความก็ได้ ขึ้นกับข้อมูลที่กำหนดให้กับแนวแกนนั้นๆ แม้ตามปกติ จะถูกใส่ลงไปอยู่แล้ว แต่สามารถปรับแต่งเพิ่มเติมได้ โดยฟังก์ชันและอาร์กิวเมนต์ที่เกี่ยวข้องกับ Label, Titte และ Tick มีดังไปนี้



27. การกำหนด Label, Title และ Tick (ต่อ)

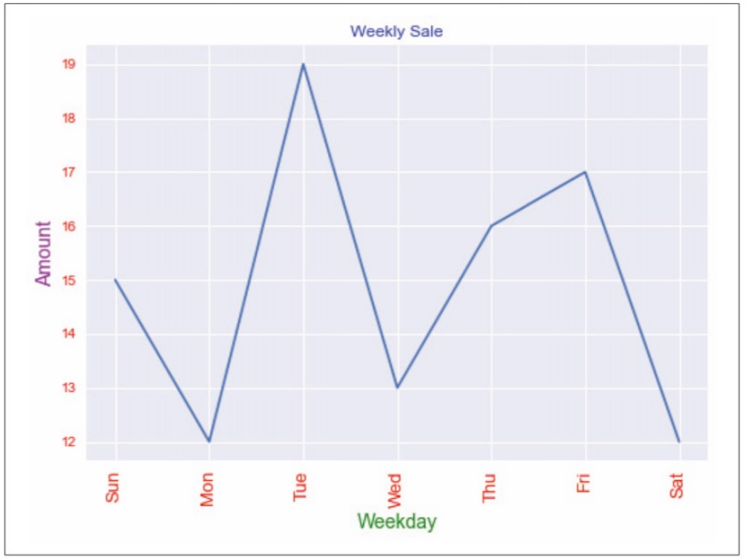
- xlabel ('ข้อความ', color, alpha, fontsize)
- ylabel (ข้อความ', color, alpha, fontsize)
- title ('ข้อความ', color, alpha, fontsize)
- xticks (rotation='vertical/horizontal/degree', color, alpha, fontsize)
- yticks (rotation='vertical/horizontal/degree', color, alpha, fontsize)
- สำหรับอาร์กิวเมนต์ rotation อาจกำหนดเป็นชื่อทิศทางคือ vertical (แนวตั้ง) หรือ horizontal (แนวนอน) หรือจะกำหนดเป็นค่ามุมว่าต้องการให้เอียงกี่องศา เช่น

```
plt.yticks(rotation='vertical', ...)
plt.xticks(rotation=45,....)
```

27. การกำหนด Label, Title และ Tick (ต่อ)

- นอกจากการกำหนดข้อความแล้ว อาจกำหนดอาร์กิวเมนต์อื่นๆ เพิ่มเติม เช่น สี (color) ความทึบ ของสี (alpha) ตามที่ได้กล่าวมาแล้ว เช่น

```
import matplotlib.pyplot as plt
plt.style.use( 'seaborn')
y = (15, 12, 19, 13, 16, 17, 12)
x=['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat']
plt.title('Weekly Sales', color='blue', alpha=0.8)
plt.ylabel('Amount', color='m', fontsize=14)
plt.xlabel('Weekday', color='g', fontsize=14)
plt.yticks(color='r')
plt.xticks(rotation='vertical', color='r', fontsize=13)
plt.plot(x, y)
plt.show()
```

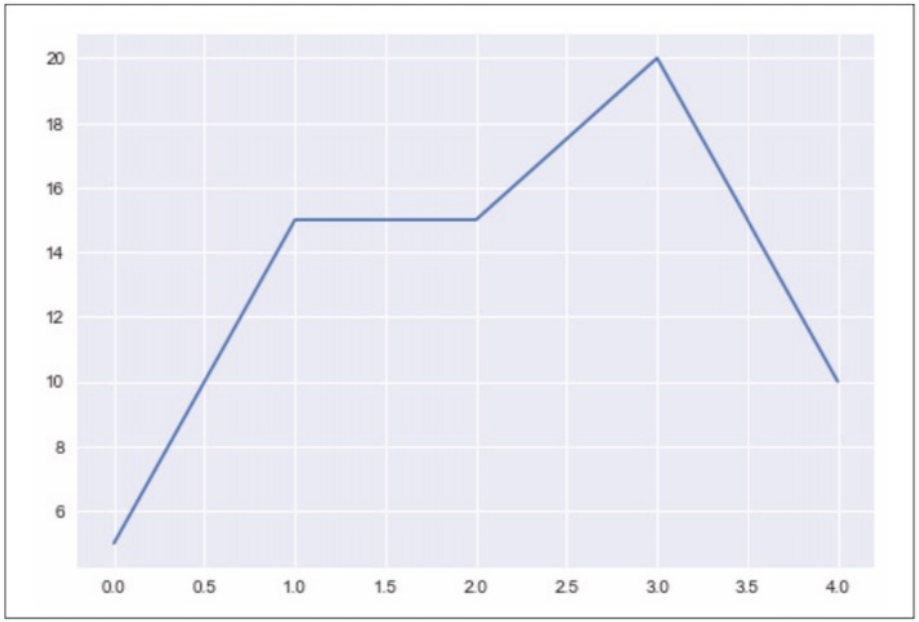


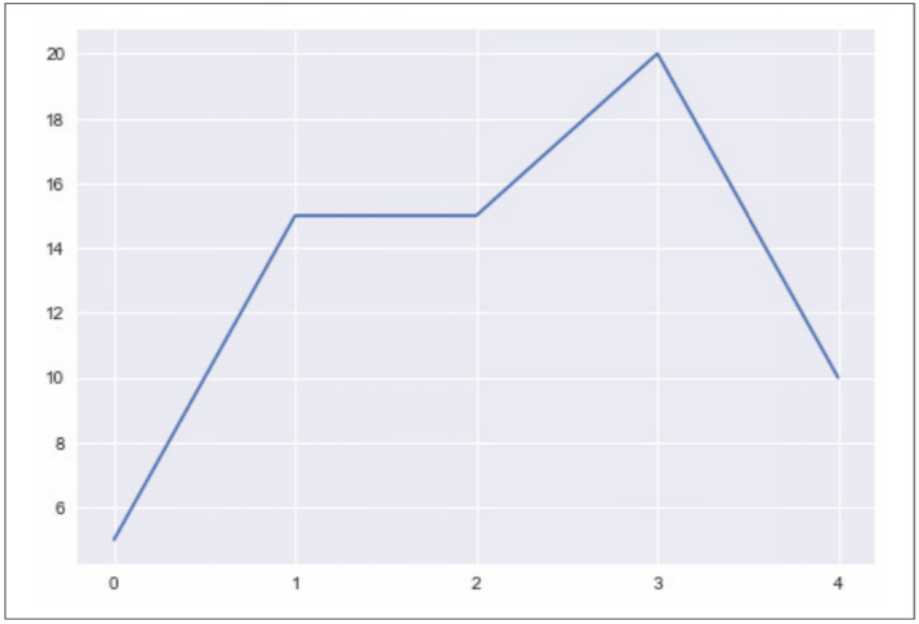
28. การกำหนดช่วงสเกล

- สามารถใช้ฟังก์ชัน xticks() และ yticks() ในการกำหนดช่วงสเกลบนแกน X และ Y ได้อีกหนึ่งกรณี หากสเกลที่แสดงบนแกน X และ/หรือ Y ไม่ตรงตามระยะที่ต้องการ โดยกำหนดเป็นช่วงข้อมูลในแบบลิสต์ เช่น ลองเปรียบเทียบโค้ดและผลลัพธ์ถัดไป ซึ่งภาพบน ไม่ระบุ xticks() ส่วนภาพล่าง กำหนดระยะสเกลให้กับ xticks()

```
import matplotlib.pyplot as plt
plt.style.use('seaborn')
y=[5,15,15,20,10]
x = [1, 3, 5, 7, 9]
plt.plot(y)
#plt.xticks([0, 1, 2, 3, 4])
plt.show()
```

28. การกำหนดช่วงสเกล (ต่อ)





28. การกำหนดช่วงสเกล (ต่อ)

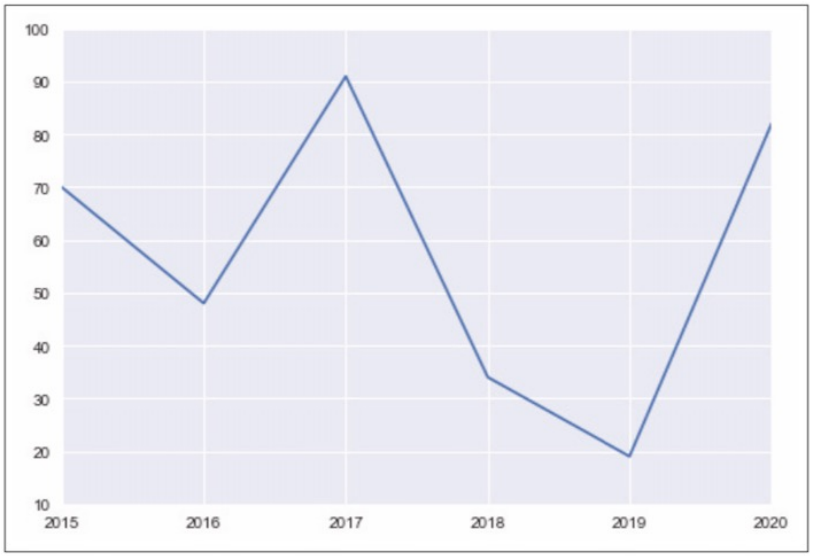
- นอกจากนี้ยังมีอีกหนึ่งกรณีที่ควรรู้เพิ่มเติมคือ หากข้อมูลที่จะนำมาวาดกราฟมีช่วงที่กว้างจนเกิน เช่น การอ่านข้อมูลมาจากที่อื่น แต่อาจสนใจข้อมูลภายในช่วงสั้นๆ เพียงบางส่วน ซึ่งลักษณะเช่นนี้สามารถใช้ฟังก์ชัน axis() เพื่อระบุขอบเขตหรือช่วงของข้อมูลที่ต้องการวาดกราฟ ดังรูปแบบต่อไปนี้

```
axis([x_start, x_stop, y_start, y_stop])
```

- x_start, x_stop คือค่าเริ่มต้นและสิ้นสุดของช่วงข้อมูลที่จะวาดกราฟในแนวแกน X
- y_start, y_stop คือค่าเริ่มต้นและสิ้นสุดของช่วงข้อมูลที่จะวาดกราฟในแนวแกน Y

28. การกำหนดช่วงสเกล (ต่อ)

- ตัวอย่าง ใน Code ต่อไปนี้ ถ้าข้อมูลที่จะนำมาวาดกราฟมีช่วงในแนวแกน X อยู่ระหว่างปี 2010-2020 แต่ ต้องการแสดงกราฟภายในปี 2015-2020 เท่านั้น ก็อาจกำหนดด้วยฟังก์ชัน axis() ดังแนวทางต่อไปนี้




```
import matplotlib.pyplot as plt
plt.style.use('seaborn' )
x= np.arange(2010, 2021)
y = np.random.randint(10, 100,11)    #ข้อมูลจริง แกน X อยู่ระหว่าง 2010 - 202020
plt.axis([2015, 2020, 10, 100])      #แกน X ให้อยู่ระหว่าง 2015 - 2020
                                     #แกน Y ให้อยู่ระหว่าง 10 - 100

plt.plot(x, y)
plt.show()
```

29. รูปแบบเส้นกราฟและกริด

- หากไม่กำหนดรูปแบบของเส้นกราฟ ก็จะได้ กราฟแบบเส้นทึบ (Solid) ดังที่ผ่านมา แต่นอกเหนือจากลักษณะดังกล่าวแล้ว สามารถเปลี่ยนรูปแบบของเส้นกราฟให้เป็นอย่างอื่นได้ โดยมีแนวทางดังนี้
- รูปแบบของเส้นกราฟ จะกำหนดด้วยอาร์กิวเมนต์ต่างๆ ดังในดารางถัดไป ให้แก่ฟังก์ชัน plot) ซึ่งอันที่น่าสนใจคือ

linestyle หรือ ls	สำหรับกำหนดรูปแบบของเส้นกราฟ เช่น เส้นทึบ หรือเส้นประ เป็นต้น
linewidth หรือ lw	กำหนดขนาด (ความหนา) ของเส้นกราฟ
color หรือ c	กำหนดสีของเส้นกราฟ
marker	กำหนดอักขระบนเส้นกราฟ ณ จุดตัดระหว่างแกน X และ Y
format string	กำหนดรูปแบบ color, linestyle และ marker ในอาร์กิวเมนต์เดียวกัน
alpha	ระดับความทึบของสี

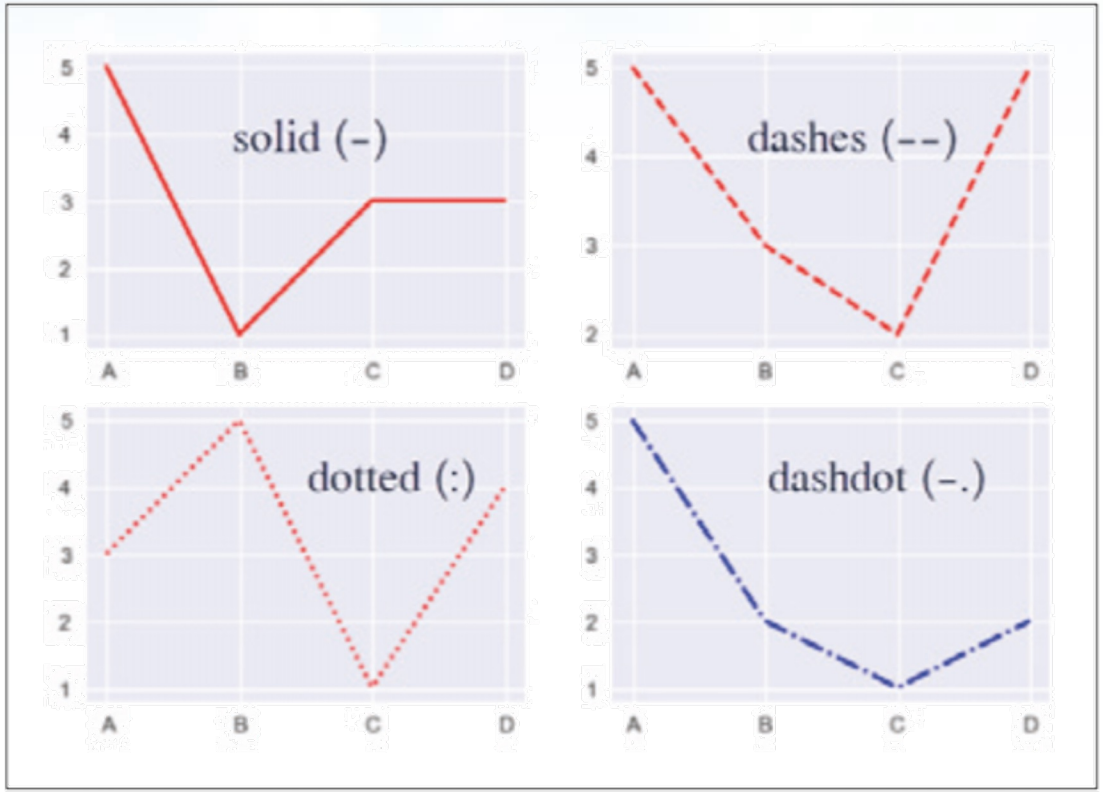
29. รูปแบบเส้นกราฟและกริด (ต่อ)

- ค่าที่กำหนดให้กับ linestyle อาจเป็นสัญลักษณ์ในรูปแบบสตริง หรือระบุชื่อโดยตรง ดังตารางซึ่งแนวทางการเขียน Code มีดังนี้
- จะใช้คีย์เวิร์ด linestyle หรือ ls ก็ได้

```
plt.plot(x, y, linestyle=' - - )
#plt.plot(x, y, 1s='dotted')
```

แบบสัญลักษณ์	แบบระบุชื่อ	คำอธิบาย
-	solid	เส้นทึบ (ค่าดีฟอลต์)
--	dashed	เส้นประ
:	dotted	จุดประ
-.	dashdot	เส้นประสลับจุด

29. รูปแบบเส้นกราฟและกริด (ต่อ)



29. รูปแบบเส้นกราฟและกริด (ต่อ)

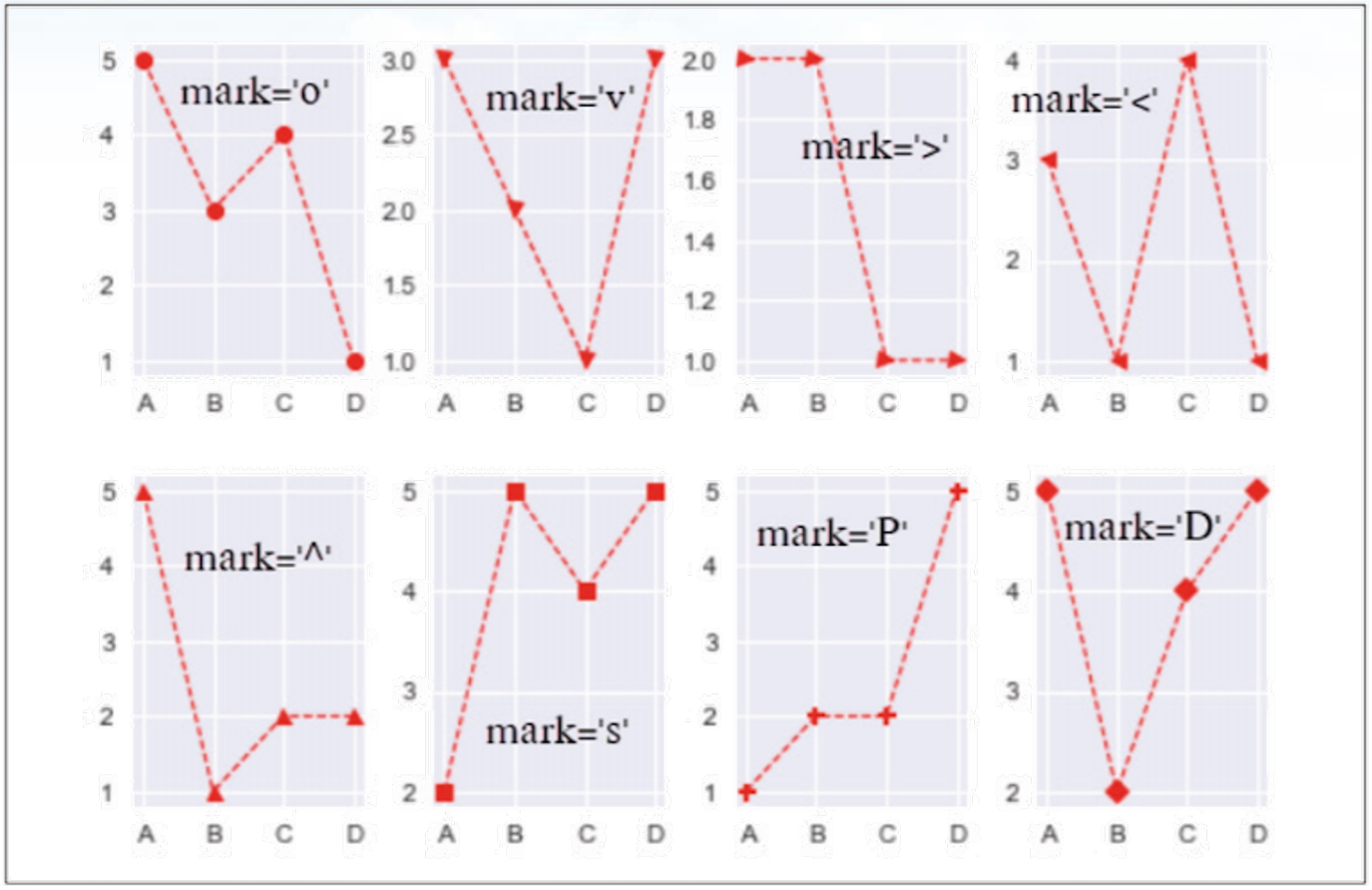
- อาร์กิวเมนต์ linewidth (หรือ lw) ให้กำหนดค่าเป็นตัวเลขตามต้องการ ส่วน/color (หรือ c) และ lpha โดยใช้รูปแบบการกำหนดสีดังที่ได้กล่าวมา เช่น

```
plt.plot(x, y, ls='--', lw=2, color='r', alpha=0.8)
```

- อาร์กิวเมนต์ marker ให้ระบุอักขระอันใดอันหนึ่งดังตารางต่อไปนี้

อักขระ	การแสดงผล	อักขระ	การแสดงผล
.	จุด	s	(เอส พิมพ์เล็ก) สี่เหลี่ยม
o	(โอ พิมพ์เล็ก) วงกลม	P	(พี พิมพ์ใหญ่) บวก
v	หัวลูกศร (สามเหลี่ยม) ชี้ลง	p	(พี พิมพ์เล็ก) ห้าเหลี่ยม
^	หัวลูกศรชี้ขึ้น	*	รูปดาว
>	หัวลูกศรชี้ไปทางขวา	h หรือ H	หกเหลี่ยม (เล็กและใหญ่)
<	หัวลูกศรชี้ไปทางซ้าย	d หรือ D	รูปเพชร (เล็กและใหญ่)

29. รูปแบบเส้นกราฟและกริด (ต่อ)



29. รูปแบบเส้นกราฟและกริด (ต่อ)

- สามารถกำหนดทั้ง color (แบบอักขระตัวเดียว), marker และ linestyle ในสตริงเดียวกัน โดยไม่ต้องระบุชื่อคีย์เวิร์ดกำกับ ซึ่งทั้งทั้ง color, marker, linestyle อาจมีเพียงอย่างเดียว หรือหรือทั้ง 3 อย่าง และสามารถเขียนสลับตำแหน่งกันได้ เช่น

```
plt.plot(x, y, 'go--’) #เส้นประ (--) สีเขียว (g) โดยใช้ o เป็น marker
plt.plot(x, y, 'y:’) #จุดประ ( : ) สีเหลือง (y)
```

29. รูปแบบเส้นกราฟและกริด (ต่อ)

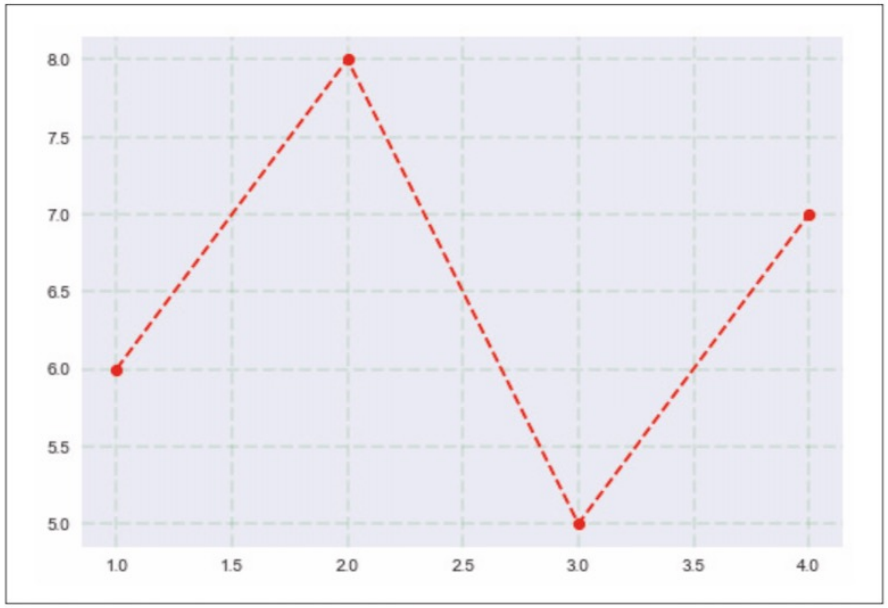
- ถ้ากำหนดสไตล์ด้วย style.use() ก็อาจมีเส้นกริด (Grid) ที่แสดงจุดตัดระหว่างแนวแกน X และ Y ด้วยฟังก์ชัน grid() ซึ่งมีอาร์กิวเมนต์ที่น่าสนใจดังนี้

grid(color, alpha, linewidth, linestyle)

- color, alpha กำหนดสีและระดับความทึบเช่นเดียวกับเส้น
- Linewidth หรือ lw คือความหนาของเส้น กำหนดค่าเป็นตัวเลขตามต้องการ
- lnestyle หรือ is คือรูปแบบเส้นกริด กำหนดค่าเช่นเดียวกับเส้นกราฟ

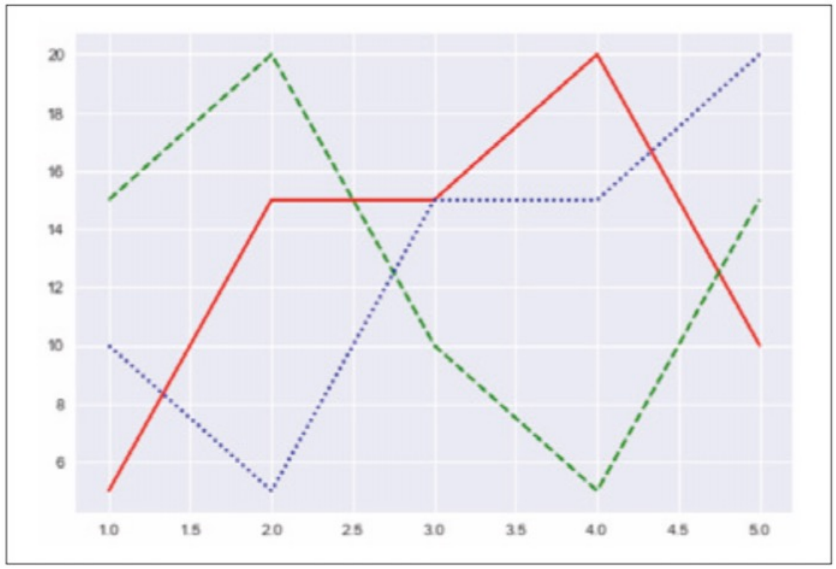
29. รูปแบบเส้นกราฟและกริด (ต่อ)

```
import matplotlib.pyplot as plt
plt.style.use('seaborn')
y = [6, 8, 5, 7]
x = [1, 2, 3, 4]
plt.plot(x, y, '--ro')
plt.grid(color='g', alpha=0.1, 1w=2, linestyle='--')
plt.show()
```



30. การวาดกราฟหลายเส้นและ Legend

- ถ้ามีข้อมูลหลายชุดอาจวาดลงบนกราฟอันเดียวกัน ซึ่งวิธีการคือ กำหนดข้อมูลแต่ละชุดแล้ววาดกราฟด้วยฟังก์ชัน plot() ตามหลักการเดิม เช่น




```
import matplotlib.pyplot as plt
plt.style.use('seaborn')
y = [5, 15, 15, 20, 10]
x = [1, 2, 3, 4, 5]
plt.plot(x, y, ls='-', c='r')
y = [15, 20, 10, 5, 15]
plt.plot(x, y, ls='--', c='g')
y = [10, 5, 15, 15, 20]
plt.plot(x, y, ls=':', c='b')
plt.show()
```

#เปลี่ยนค่าในแนวแกน Y ใหม่ ส่วนแกน X ใช้ค่าเดิม

#เปลี่ยนค่าในแนวแกน Y ใหม่ ส่วนแกน X ใช้ค่าเดิม

30. การวาดกราฟหลายเส้นและ Legend (ต่อ)

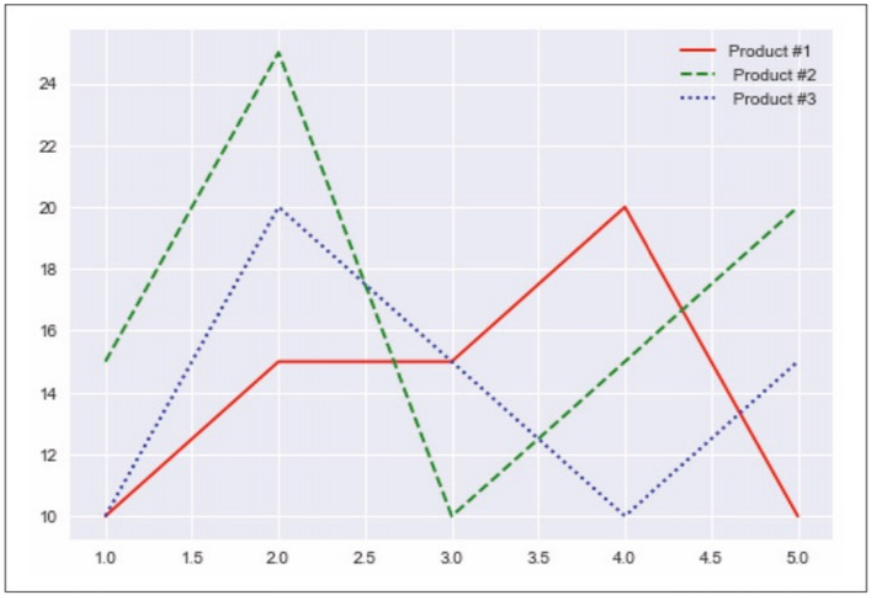
- กรณีที่มีกราฟหลายเส้นอาจเพิ่ม Legend เพื่อให้ทราบว่ากราฟแต่ละเส้นเป็นข้อมูลอะไร โดยใช้หลักการดังนี้
- ที่ฟังก์ชัน plot() ของกราฟแต่ละเส้น ให้เพิ่มอาร์กิวเมนต์ label เพื่อระบุชื่อข้อมูลของกราฟเส้นนั้น เช่น

```
plt.plot(x, y, label='graph 1')
```

30. การวาดกราฟหลายเส้นและ Legend (ต่อ)

- ก่อนแสดงกราฟให้เรียกฟังก์ชัน pit.legend() ซึ่งฟังก์ชันนี้มีอาร์กิวเมนต์ที่สำคัญคือ loc สำหรับระบุตำแหน่งที่จะแสดง Legend แต่โดยส่วนใหญ่มักจะเลือก ‘best’
- สำหรับแนวทางการวาดกราฟพร้อมกำหนด Legend มีดังนี้

```
import matplotlib.pyplot as plt
plt.style.use('seaborn')
y=[10,15,15,20,10]
x = [1, 2, 3, 4, 5]
plt.plot(x, y, 1s='-', c='r', label='Product #1')
y=[15,25,10,15,20]
plt.plot(x, y, 1s='--', c='g', label=' Product #2’)
y=[10,20,15,10,15]
plt.plot(x, y, ls=':', c='b', label=' Product #3')
plt.legend(loc=' best')
plt.show ( )
```



31. การวาดกราฟแบบ Scatter

- Scatter เป็นการวาดกราฟแบบกระจายกลุ่มข้อมูลของ 2 ตัวแปรในแนวแกน X และ Y โดยทั่วไปแล้วแบบ Scatter มักใช้เพื่อเปรียบเทียบความสัมพันธ์ระหว่างข้อมูลของตัวแปรหนึ่งกับอีกตัวแปรหนึ่ง ซึ่งใน Machine Leaming ต้องนำกราฟแบบ Scatter ไปใช้อยู่บ่อยๆ โดยแนวทางการวาดกร
  - กราฟชนิดนี้ต้องกำหนดข้อมูลในแนวแกน X และ Y ให้เป็นแบบรายการ เช่น ลิสต์ ทูเพิล หรืออาร์เรย์ และต้องมีจำนวนเท่ากันทั้งสองแนวแกน
  - วาดกราฟด้วยฟังก์ชัน plt.scatter () โดยอาร์กิวเมนต์พื้นฐานก็คล้ายกับกราฟแบบ Linear แล้ว ซึ่งอันที่นิยมนำมาใช้กับกราฟชนิดนี้อยู่บ่อยๆ เช่น color, marker เป็นต้น หรืออาจปรับขนาดของ maker ด้วยอาร์กิวเมนต์ s เช่น s=100

31. การวาดกราฟแบบ Scatter (ต่อ)

```
import matplotlib.pyplot as plt
plt.style.use('seaborn')
y = np.random.randint(1, 101, 20)
x = np.arange(1, 21)
plt.scatter(x, y, color='g', marker='o', s=80)
plt.yticks(np.arange(0, 101, 10))
plt.xticks(x)
plt.show()
```

